Survival Guide

Using QBasic:

- To activate the QBasic menu bar, press Alt.
- To activate menus and commands, press the highlighted letter.
- To move between menus and commands, use the direction keys.
- To get help on a selected keyword, command, or dialog box, press F1.
- To exit Help, press Esc.

Browsing the QBasic Help system:

- To select one of the following topics, press the Tab key or the first letter of the topic. Then press the Enter key to see:

«Index»	The Index for QBasic Help
«Contents»	The Table of Contents for QBasic Help topics
«Using Help»	Information on using QBasic Help

Tip: These topics are also available from the Help menu.

Using Help

Using QBasic Help:

- To get help on a Basic keyword, place the cursor on it and press F1 or click the right mouse button.

- To get help on a QBasic menu, command, or dialog box, place the cursor on the menu item or <Help> button and press F1.

- To view QBasic Help topics, press Alt+H, then press the highlighted letter to choose a command.

- To move the cursor into the Help window, press Shift+F6.

- To scroll the help information, press PgDn or PgUp.

- To copy help information (such as a programming example) to the View window, use the commands on the QBasic Edit menu.

- To close the Help window, press Esc.

In the Help window, you can display information on:

- Help topics (identified by highlighted arrowheads « »)

- Basic keywords

To move the cursor to a Help topic, press Tab or the first letter of the topic. To display the topic or keyword information, place the cursor anywhere on the topic or keyword and press F1 or click the right mouse button.

QBasic saves the last 20 Help topics you viewed. To review them press Alt+F1 or click the <Back> button repeatedly.

Table of Contents

«Contents» «Index» «Back»

+Orientation-------------------------+ +Keys----------------------------+
«Using Help»		«Shortcut Keys Summary»
«Using Menus and Commands»		«Editing Keys»
«Using a Dialog Box»		«View and Search Keys»
«Syntax Conventions»		«Run and Debug Keys»
+-------------------------------------+ | «Help Keys» |
+Using QBasic-------------------------+ +-------------------------------+
| «QBasic Command Line» | +Quick Reference----------------+
«Basic Character Set»		«ASCII Character Codes»
«Keywords by Programming Task»		«Keyboard Scan Codes»
«QBasic Environment Limits»		«Run-Time Error Codes»
«Version Differences»		«Copyright and Trademarks»
«Converting BASICA Programs»	+-------------------------------+
«Beyond QBasic»
+-------------------------------------+

Copyright and Trademarks

«Contents» «Index» «Back»

Information in this document is subject to change without notice and does not represent a commitment on the part of Microsoft Corporation. The software and/or files described in this document are furnished under a license agreement or nondisclosure agreement. The software and/or files may be used or copied only in accordance with the terms of the agreement. The purchaser may make one copy of the software for backup purposes. No part of this online help system may be reproduced or transmitted in any form or by any means, electronic or mechanical, including photocopying, recording, or information storage and retrieval systems, for any purpose, without the written permission of Microsoft Corporation.

RESTRICTED RIGHTS. Use by the U.S. Government subject to restrictions of (c) (1) (ii) of DFARS 252.227-7013.

(C) Copyright Microsoft Corporation, 1987-1992.
All rights reserved.

Microsoft, MS, MS-DOS, Microsoft Press, and GW-BASIC are registered trademarks.

Microsoft documentation uses the term "DOS" to refer to both the MS-DOS and IBM Personal Computer DOS operating systems. The name of a specific operating system is used when it is necessary to note features that are unique to the system.

AT&T is a registered trademark of American Telephone and Telegraph Company.

Compaq is a registered trademark of Compaq Computer Corporation.

Hercules is a registered trademark of Hercules Computer Technology.

IBM is a registered trademark of the International Business Machines Corporation.

Olivetti is a registered trademark of Ing. C. Olivetti.

WordStar is a registered trademark of MicroPro International Corporation.

Syntax Conventions

«Contents» «Index» «Back»

KEYWORDS	Items in capital letters are Basic keywords. Keywords are a required part of the statement syntax, unless they are enclosed in brackets.
placeholders	Items in lowercase are placeholders for information you must supply in the statement (such as a filename$). The QBasic syntax uses data-type suffixes for placeholders that must be a specific data type. Placeholders that can be more than one data type do not have data-type suffixes.
[optional item]	Items inside square brackets are optional.
{choice1 | choice2}	Braces and a vertical bar indicate a choice between two or more items. You must use one of the items in the statement unless the braces are enclosed in square brackets.
item, item, ...	A horizontal three-dot ellipsis means more of the preceding items can be used in a single-line statement.
Beginning keyword . . . Ending keyword	A vertical three-dot ellipsis is used to describe multiline statements (or block-structured statements). It means that other statements can be used between the beginning and the end of the block.

Using Menus and Commands

Use the menu bar at the top of the screen to select menus and commands.

Menu action	With a mouse...	With a keyboard...
Display a menu.	Move the mouse pointer to the menu name, then press and release ("click") the mouse button.	Press Alt to highlight menu letters, then press the letter key for the menu you want to display.
Choose a command.	Click the command name.	Press the letter key that matches the highlighted letter on the command.
Cancel a command.	Click outside the menu.	Press Esc.

Tip: Check the reference bar at the bottom of the screen for help on your current task. Choose any item in angle brackets by clicking the item.

Using a Dialog Box

«Contents» «Index» «Back»

QBasic displays dialog boxes when you choose commands that have options.

Dialog box action	With a mouse...	With a keyboard...
Select a text box for input.	Move the mouse pointer to the text box, then press and release ("click") the mouse button.	Press Tab until the cursor is in the text box.
Choose an item from a list box.	Double-click the item.	Highlight the item by pressing the direction keys, then press Enter.
Choose an option.	Click the option.	Move to an option with the direction keys.
Turn a check box on or off.	Click the check box.	Use Tab to move the cursor to the check box, then press the Spacebar.
Choose a command button.	Click the command button.	Use Tab to move the cursor to the command button, then press Enter.

QBasic Command Line

«Contents» «Index» «Back»

These options can be typed on the DOS command line following the QBasic command:

QBASIC [/B] [/EDITOR] [/G] [/H] [/MBF] [/NOHI] [[/RUN] sourcefile]

Option	Description

/B	Allows the use of a composite (monochrome) monitor with a color graphics card. The /B option displays QBasic in monochrome if you have a color monitor.
/EDITOR	Invokes the MS-DOS Editor text editor. Can be abbreviated as /ED.
/G	Sets QBasic to update a CGA screen as fast as possible (works only with machines using CGA monitors). If you see snow (dots flickering on the screen) when QBasic updates your screen, your hardware cannot fully support this option. If you prefer a clean screen, restart QBasic without the /G option.
/H	Displays the maximum number of lines possible on your hardware.
/MBF	Causes the QBasic conversion functions (CVS, CVD, MKS$, MKD$) to treat IEEE-format numbers as Microsoft-Binary-format numbers.
/NOHI	Allows the use of a monitor that does not support high intensity. Not for use with Compaq laptop computers.
sourcefile	Names the file to load when QBasic starts. To load a file created with GW-BASIC or BASICA, the file must be saved from GW-BASIC or BASICA with the ,A option.
/RUN sourcefile	Causes QBasic to load and run a program file before displaying it.

Basic Character Set

The Microsoft Basic character set includes alphabetic characters (A-Z, a-z), numeric characters (0-9 and A-F or a-f for hexadecimal numbers), and special characters. Some characters have special meanings in Basic:

Data-Type Suffixes			
!	Single-precision	%	Integer
#	Double-precision	&	Long-integer
$	String		

Mathematical Operators		Special	
*	Multiplication symbol	'	Comment line (single quote)
-	Minus sign	;	Controls PRINT and INPUT

/ Division symbol (slash)	statement output
= Relational operator or assignment symbol	, Controls PRINT and INPUT statement output
> Greater than	: Separates multiple statements on a single line
+ Plus sign	? INPUT statement prompt
. Decimal point	_ Line continuation underscore (reserved for compatibility with other versions of Basic but not supported by QBasic)
< Less than	
\\ Integer division symbol (backslash)	
^ Exponentiation symbol (up arrow or caret)	

Shortcut Keys Summary

«Contents» «Index» «Back»

Selecting	
Characters/lines	Shift+Arrow
Words	Shift+Ctrl+Arrow

Inserting	
Switch to insert/overstrike	Ins
Line above	Home,Ctrl+N
Line below	End+Enter
From Clipboard	Shift+Ins

Finding	
Search for text	Ctrl+Q,F
Repeat find	F3

Getting Help	
On QBasic Help	Shift+F1
On keywords or topics	F1

Deleting	
Cut current line	Ctrl+Y
Cut to end of line	Ctrl+Q,Y
Cut selected text	Shift+Del
Erase selected text	Del

Copying	
To Clipboard	Ctrl+Ins

Debugging	
View output screen	F4
Continue running	F5
Execute to cursor	F7
Toggle breakpoint	F9
Single step	F8
Procedure step	F10

Run and Debug Keys

«Contents» «Index» «Back»

Start program execution from beginning	Shift+F5
Continue program execution from current statement	F5
Execute program to current cursor position	F7

Execute next program statement as a single step	F8
Single step, tracing around procedure calls	F10
Set or clear a breakpoint	F9

View and Search Keys

Switch between the output screen and the View window	F4
Display a list of loaded SUB procedures	F2
Display the next procedure	Shift+F2
Display the previous procedure	Ctrl+F2
Make the next window the active window	F6
Make the previous window the active window	Shift+F6
Switch between multiple windows and full-screen active window	Ctrl+F10
Increase size of active window	Alt+Plus
Decrease size of active window	Alt+Minus
Repeat find for same text	Ctrl+L or F3
Search for text	Ctrl+Q,F
Search for and replace text	Ctrl+Q,A

Help Keys

View Help for the item where the cursor is located	F1 (or click the right mouse button)
Exit Help	Esc
View Using Help	Shift+F1
Display the Help menu	Alt+H
Move cursor to next Help topic	Tab
Move cursor to previous Help topic	Shift+Tab
Move cursor to next topic with starting character	character
Move cursor to previous topic with starting character	Shift+character

View previous Help topic (can repeat up to 20 times)	Alt+F1 (or click the <Back> button)

View the next topic in the Help file	Ctrl+F1
View the previous topic in the Help file	Shift+Ctrl+F1

Editing Keys

«Contents» «Index» «Back»

Cursor-Movement Keys	**QBasic**	**WordStar**
Character left	Left Arrow	Ctrl+S
Character right	Right Arrow	Ctrl+D
Word left	Ctrl+Left Arrow	Ctrl+A
Word right	Ctrl+Right Arrow	Ctrl+F
Line up	Up Arrow	Ctrl+E
Line down	Down Arrow	Ctrl+X
First indentation level of current line	Home	
Beginning of current line		Ctrl+Q,S
Beginning of next line	Ctrl+Enter	Ctrl+J
End of line	End	Ctrl+Q,D
Top of window		Ctrl+Q,E
Bottom of window		Ctrl+Q,X
Move to next window	F6	
Increase size of active window	Alt+Plus	
Decrease size of active window	Alt+Minus	

Text-Scrolling Keys	**QBasic**	**WordStar**
Line up	Ctrl+Up Arrow	Ctrl+W
Line down	Ctrl+Down Arrow	Ctrl+Z
Page up	PgUp	Ctrl+R
Page down	PgDn	Ctrl+C
Left one window	Ctrl+PgUp	
Right one window	Ctrl+PgDn	
Set bookmarks (maximum of 4)	Ctrl+K,0-3	
Go to set bookmarks	Ctrl+Q,0-3	

Text-Selection Keys	**QBasic**
Character left	Shift+Left Arrow
Character right	Shift+Right Arrow
Word left	Shift+Ctrl+Left Arrow
Word right	Shift+Ctrl+Right Arrow
Current line	Shift+Down Arrow
Line above	Shift+Up Arrow
Screen up	Shift+PgUp
Screen down	Shift+PgDn

To beginning of file	Shift+Ctrl+Home	
To end of file	Shift+Ctrl+End	

Insert and Copy Keys	**QBasic**	**WordStar**
Switch between insert and overstrike modes	Ins	Ctrl+V
Copy selected text to Clipboard	Ctrl+Ins	
Delete selected text and copy it to Clipboard	Shift+Del	
Delete current line and copy it to Clipboard	Ctrl+Y	
Delete to end of line and copy it to Clipboard		Ctrl+Q,Y
Paste the contents of the Clipboard	Shift+Ins	
Insert a blank line below the cursor position	End,Enter	
Insert a blank line above the cursor position		Home,Ctrl+N
Insert special characters	Ctrl+P,Ctrl+key	

Delete Keys	**QBasic**	**WordStar**
Delete one character to left of the cursor	Backspace	Ctrl+H
Delete one character at the cursor	Del	Ctrl+G
Delete the rest of the word the cursor is on		Ctrl+T
Delete selected text	Del	Ctrl+G
Delete leading spaces from selected lines	Shift+Tab	

Keywords by Programming Task

«Contents» «Index» «Back»

Programming task	Keywords included in this list
Control program flow	DO...LOOP, END, EXIT, FOR...NEXT, IF...THEN...ELSE, GOSUB...RETURN, GOTO, ON...GOSUB, ON...GOTO, SELECT CASE, STOP, SYSTEM
Declare constants and variables	CONST, DATA, DIM, ERASE, OPTION BASE, READ, REDIM, REM, RESTORE, SWAP, TYPE...END TYPE
Define and call Basic procedures	CALL, DECLARE, EXIT, FUNCTION, RUN, SHELL, SHARED, STATIC, SUB
Device input/output	CLS, CSRLIN, INKEY$, INP, INPUT, KEY (Assignment), LINE INPUT, LOCATE, LPOS, LPRINT, LPRINT USING, OPEN COM, OUT, POS, PRINT, PRINT USING, SPC, SCREEN Function, TAB, VIEW PRINT, WAIT, WIDTH

Display graphic images	CIRCLE, COLOR, GET (Graphics), LINE, PAINT, PALETTE, PCOPY, PMAP, POINT, PRESET, PSET, PUT (Graphics), SCREEN Statement, VIEW, WINDOW
DOS file system commands	CHDIR, KILL, MKDIR, NAME, RMDIR
File input/output	CLOSE, EOF, FILEATTR, FREEFILE GET (File I/O), INPUT, INPUT$, LINE INPUT, LOC, LOCK, LOF, OPEN, PUT (File I/O), SEEK Function, SEEK Statement, UNLOCK, WRITE
Manage memory	CLEAR, FRE, PEEK, POKE
Manipulate strings	ASC, CHR$, HEX$, INSTR, LCASE$, LEFT$, LEN, LSET, LTRIM$, MID$ Function, MID$ Statement, OCT$, RIGHT$, RSET, RTRIM$ SPACE$, STR$, STRING$, UCASE$, VAL
Perform mathematical calculations	ABS, ASC, ATN, CDBL, CINT, CLNG, COS, CSNG, CVDMBF, CVSMBF, EXP, INT, LOG, RANDOMIZE, RND, SGN, SIN, SQR, TAN, TIME$ Function
Set traps for events and errors	COM, ERDEV, ERDEV$, ERL, ERR, ERROR, KEY (Event Trapping), ON COM, ON ERROR, ON KEY, ON PEN, ON PLAY, ON STRIG, ON TIMER, PEN, PLAY (Event Trapping), RESUME, RETURN, STRIG, TIMER Function, TIMER Statement

Version Differences

«Contents» «Index» «Back»

Language Features	BASICA/GW-BASIC	QBasic
Line numbers	Required	Optional
Block IF/THEN/ELSE/END IF	No	Yes
Procedural blocks (SUB and FUNCTION)	No	Yes
User-defined types (records)	No	Yes
SELECT CASE	No	Yes

DO WHILE/UNTIL, LOOP WHILE/UNTIL	No	Yes
LONG (32-bit) integers	No	Yes
IEEE-format numbers	No	Yes
Constants	No	Yes
Fixed-length strings	No	Yes
VGA screen modes	No	Yes
Hercules/Olivetti support	No	Yes
Recursion	No	Yes
Cassette tape support	Yes	No
Code + data capacity	64K	160K

Debugging

Single step	No	Yes
Procedure step	No	Yes
Trace animation	No	Yes
Breakpoints	No	Yes
Edit and continue	No	Yes

Editing

Instant syntax checking	No	Yes
Diagnostic error messages	No	Yes
Instant help on errors	No	Yes
Multiple windows	No	Yes
Mouse support	No	Yes
Menu interface	No	Yes
Wordstar compatible	No	Yes
Code organization	Linear	Modular
Document mode for text editing	No	Yes
Search/find/replace	No	Yes
Online reference	None	Full

See Also «Differences from BASICA» «Unsupported Keywords»

ASCII Character Codes

«Contents» «Index» «Back»

Regular ASCII Chart (character codes 0 - 127)

000	(nul)	016 »	(dle)	032	sp	048	0	064	@	080	P	096	`	112	p
001 ?	(soh)	017 «	(dc1)	033	!	049	1	065	A	081	Q	097	a	113	q
002 ?	(stx)	018 ?	(dc2)	034	"	050	2	066	B	082	R	098	b	114	r
003 ?	(etx)	019 ?	(dc3)	035	#	051	3	067	C	083	S	099	c	115	s
004 ?	(eot)	020 ?	(dc4)	036	$	052	4	068	D	084	T	100	d	116	t
005 ?	(enq)	021 ?	(nak)	037	%	053	5	069	E	085	U	101	e	117	u
006 ?	(ack)	022 ?	(syn)	038	&	054	6	070	F	086	V	102	f	118	v
007 ?	(bel)	023 ?	(etb)	039	'	055	7	071	G	087	W	103	g	119	w
008 ?	(bs)	024 ?	(can)	040	(	056	8	072	H	088	X	104	h	120	x
009	(tab)	025 ?	(em)	041	)	057	9	073	I	089	Y	105	i	121	y

010 (lf)	026 (eof)	042 *	058 :	074 J	090 Z	106 j	122 z		
011 ? (vt)	027 ? (esc)	043 +	059 ;	075 K	091 [	107 k	123 {		
012 ? (np)	028 ? (fs)	044 ,	060 <	076 L	092 \\	108 l	124 |		
013 (cr)	029 ? (gs)	045 -	061 =	077 M	093]	109 m	125 }		
014 ? (so)	030 ? (rs)	046 .	062 >	078 N	094 ^	110 n	126 ~		
015 ? (si)	031 ? (us)	047 /	063 ?	079 O	095 _	111 o	127 ?		

Extended ASCII Chart (character codes 128 - 255)

128 Ç	143 Å	158 ₧	172 ¼	186 ╣	200 ╚	214 ╓	228 Σ	242 ≥	
129 ü	144 É	159 ƒ	173 ¡	187 ╗	201 ╔	215 ╫	229 σ	243 ≤	
130 é	145 æ	160 á	174 «	188 ╝	202 ╩	216 ╪	230 µ	244 ⌠	
131 â	146 Æ	161 í	175 »	189 ╜	203 ╦	217 ┘	231 τ	245 ⌡	
132 ä	147 ô	162 ó	176 ░	190 ╛	204 ╠	218 ┌	232 Φ	246 ÷	
133 à	148 ö	163 ú	177 ▒	191 ┐	205 ═	219 █	233 Θ	247 ≈	
134 å	149 ò	164 ñ	178 ▓	192 └	206 ╬	220 ▄	234 Ω	248 °	
135 ç	150 û	165 Ñ	179 |	193 ┴	207 ╧	221 ▌	235 δ	249 ·	
136 ê	151 ù	166 ª	180 ┤	194 ┬	208 ╨	222 ▐	236 ∞	250 ·	
137 ë	152 ÿ	167 º	181 ╡	195 ├	209 ╤	223 ▀	237 φ	251 √	
138 è	153 Ö	168 ¿	182 ╢	196 -	210 ╥	224 α	238 ε	252 ⁿ	
139 ï	154 Ü	169 ⌐	183 ╖	197 ┼	211 ╙	225 ß	239 ∩	253 ²	
140 î	155 ¢	170 ¬	184 ╕	198 ╞	212 ╘	226 Γ	240 ≡	254 ■	
141 ì	156 £	171 ½	185 ╣	199 ╟	213 ╒	227 π	241 ±	255	
142 Ä	157 ¥								

Keyboard Scan Codes

«Contents» «Index» «Back»

Key	Code	+	Key	Code	+	Key	Code
		+			+		
Esc	1	+	A	30	+	Caps Lock	58
! or 1	2	+	S	31	+	F1	59
@ or 2	3	+	D	32	+	F2	60
# or 3	4	+	F	33	+	F3	61
$ or 4	5	+	G	34	+	F4	62
% or 5	6	+	H	35	+	F5	63
^ or 6	7	+	J	36	+	F6	64
& or 7	8	+	K	37	+	F7	65
* or 8	9	+	L	38	+	F8	66
(or 9	10	+	: or ;	39	+	F9	67
) or 0	11	+	" or '	40	+	F10	68
_ or -	12	+	~ or `	41	+	F11	133
+ or =	13	+	Left Shift	42	+	F12	134
Bksp	14	+	| or \\	43	+	NumLock	69
Tab	15	+	Z	44	+	Scroll Lock	70
Q	16	+	X	45	+	Home or 7	71
W	17	+	C	46	+	Up or 8	72
E	18	+	V	47	+	PgUp or 9	73
R	19	+	B	48	+	Gray -	74
T	20	+	N	49	+	Left or 4	75
Y	21	+	M	50	+	Center or 5	76
U	22	+	< or ,	51	+	Right or 6	77
I	23	+	> or .	52	+	Gray +	78
O	24	+	? or /	53	+	End or 1	79

P	25	+	Right Shift	54	+	Down or 2	80
{ or [	26	+	Prt Sc or *	55	+	PgDn or 3	81
} or]	27	+	Alt	56	+	Ins or 0	82
Enter	28	+	Spacebar	57	+	Del or .	83
Ctrl	29	+			+		

Run-Time Error Codes

Code	Message	+	Code	Message
		+		
1	NEXT without FOR	+	37	Argument-count mismatch
2	Syntax error	+	38	Array not defined
3	RETURN without GOSUB	+	40	Variable required
4	Out of DATA	+	50	FIELD overflow
5	Illegal function call	+	51	Internal error
6	Overflow	+	52	Bad file name or number
7	Out of memory	+	53	File not found
8	Label not defined	+	54	Bad file mode
9	Subscript out of range	+	55	File already open
10	Duplicate definition	+	56	FIELD statement active
11	Division by zero	+	57	Device I/O error
12	Illegal in direct mode	+	58	File already exists
13	Type mismatch	+	59	Bad record length
14	Out of string space	+	61	Disk full
16	String formula too complex	+	62	Input past end of file
17	Cannot continue	+	63	Bad record number
18	Function not defined	+	64	Bad file name
19	No RESUME	+	67	Too many files
20	RESUME without error	+	68	Device unavailable
24	Device timeout	+	69	Communication-buffer overflow
25	Device fault	+	70	Permission denied
26	FOR without NEXT	+	71	Disk not ready
27	Out of paper	+	72	Disk-media error
29	WHILE without WEND	+	73	Feature unavailable
30	WEND without WHILE	+	74	Rename across disks
33	Duplicate label	+	75	Path/File access error
35	Subprogram not defined	+	76	Path not found

QBasic Environment Limits

This section lists the limits for the following items in the QBasic environment:

«Name, String, and Number Limits»

«Array Limits»
«Procedure and File Limits»

See Also «Beyond QBasic»

Name, String, and Number Limits

«Contents» «Index» «Back»

	Maximum	Minimum
Variable-name length	40 characters	1 character
String length	32,767 characters	0 characters
Integers	32,767	-32,768
Long integers	2,147,483,647	-2,147,483,648
Single-precision numbers:		
Positive	3.402823E+38	2.802597E-45
Negative	-2.802597E-45	-3.402823E+38
Double-precision numbers:		
Positive	1.79769313486231D+308	4.940656458412465D-324
Negative	-4.940656458412465D-324	-1.79769313486231D+308

Array Limits

«Contents» «Index» «Back»

	Maximum	Minimum
Array size (all elements):		
Static	65,535 bytes (64K)	1 byte
Dynamic	65,535 bytes (64K)	
Number of dimensions allowed	60	1
Dimensions allowed if unspecified	8	1
Array subscript value	32,767	-32,768

Note: The maximum range between array subscript values is 32,767.

Procedure and File Limits

«Contents» «Index» «Back»

	Maximum	Minimum

	==========	==========
Procedure size	64K	0
Number of arguments passed	60	0
Data file numbers	255	1
Data file record number	2,147,483,647	1
Data file record size	32K	1 byte
Data file size	Available disk space	0
Path names	127 characters	1 character
Error message numbers	255	1

Converting BASICA Programs

Remember the following points when converting BASICA programs to QBasic:

- Save the BASICA program in ASCII format. QBasic cannot use programs that are saved in BASICA's binary format.
- Use the /MBF command option when starting QBasic. The /MBF option enables QBasic to read and write BASICA data files.
- Change CALL statements to CALL ABSOLUTE. The BASICA CALL statement is equivalent to QBasic's CALL ABSOLUTE.

In addition, you may want to use REMLINE.BAS to remove unnecessary line numbers from the programs you are converting. QBasic does not require line numbers and removing them can improve a program's readability.

See Also «Differences from BASICA» «Unsupported Keywords» «Version Differences»

Beyond QBasic

QBasic is an easy-to-use environment for creating Basic programs.
The following table lists some solutions if you need to go beyond QBasic.

If you need	You can
==========	==========
Fast program execution	Use a Basic compiler (such as Microsoft

	Visual Basic for MS-DOS) to translate your Basic code into native machine code.
To build programs that require over 160K of memory	Use a Basic development environment that supports creating large programs.
To distribute your programs	Use Microsoft Visual Basic for MS-DOS or another Basic compiler to create stand-alone applications from your QBasic code. This allows you to distribute your programs and eliminates the need for QBasic to be present.
To add mouse control to your application or create a standard user interface.	Use Microsoft Visual Basic for MS-DOS, which has built-in mouse support and tools to help you create an attractive user interface.

Contact your local software dealer if you want more information about Basic programming products. If you want to learn more about programming with Basic, consult the following Microsoft Press books or other books for information:

- "Running QBasic" by Michael Halvorson and David Rygmyr (1991).
- "QBasic: Programmer's Quick Reference" by Kris Jamsa (1991).
- "The Waite Group's QBasic Reference Plus" by Waite, Prata, et al. (1991).
- "Do It Yourself Visual Basic for MS-DOS" by WIlliam J. Orvis (1992).
- "Microsoft Guide to Visual Basic for MS-DOS Programming" by Peter G. Aitken (1992).
- "Peter Norton's Guide to Visual Basic for MS-DOS Programming" by Steve Holzer and the Peter Norton Computing Group (1992).
- "Teach Yourself Visual Basic for MS-DOS" by Chuck Butkus(1992).
- "Visual Basic for MS-DOS by Example" by D.F. Scott (1992).
- "Visual Basic for MS-DOS Inside and Out" by David I. Schneider and Gary Cornell (1992).
- "Visual Basic for MS-DOS Uncovered" by Bryon Scott, et al.(1992).

Unsupported Keywords

The following QuickBasic keywords are not supported in QBasic:

ALIAS	EVENT	LOCAL	SETMEM
BYVAL	$INCLUDE	SADD	SIGNAL
CDECL	Int86	Interrupt	UEVENT
COMMAND$	Int86X	InterruptX	

The following GW-BASIC/BASICA keywords are not supported in QBasic:

AUTO	EDIT	MERGE	RENUM
CONT	LIST	MOTOR	SAVE
DEF USR	LLIST	NEW	USR
DELETE	LOAD		

See Also «Converting BASICA Programs» «Differences from BASICA» «Version Differences»

Differences from BASICA

«Contents» «Index» «Back»

The following keywords behave differently in QBasic than in BASICA:

BSAVE, BLOAD QBasic does not support the cassette device with BSAVE and BLOAD, and memory locations may be different.

CALL ABSOLUTE Assembly language programs invoked from BASICA that have string arguments must be revised for QBasic because QBasic string descriptors are 4 bytes long. The 4 bytes are the low byte and high byte of the string length followed by the low byte and high byte of the string address.

CHAIN QBasic does not allow you to specify a line number with CHAIN and does not support the ALL, MERGE, or DELETE options in BASICA.

COMMON, DECLARE QBasic does not allow executable statements to appear before COMMON and DECLARE statements.

DEFtype If BASICA encounters a variable without an explicit type (indicated by !, #, &, $, or %), it uses the default type set by the most recent DEFtype statement. For example, the type of the variable IFLAG changes from integer to single precision in the following BASICA code fragment:

```
10  DEFINT I
20  PRINT IFLAG
30  DEFSNG I
40  GOTO 20
```

In QBasic, a variable's type cannot be changed after it appears in a program.

DIM All arrays in BASICA are dynamic because BASICA always

allocates array storage while a program runs. In QBasic, DIM can declare either a static or dynamic array.

DRAW, PLAY

DRAW and PLAY statements in BASICA that use variables or other command strings within command strings must be modified for QBasic. Use the VARPTR$ function, which returns a string representation of the variable's address:

BASICA Statement	QBasic Equivalent
DRAW "Xcmdstring$"	DRAW "X" + VARPTR$ (cmdstring$)
DRAW "TA = angle"	DRAW "TA =" + VARPTR$(angle)

EOF

When you direct input from a file to a BASICA program, EOF(0) returns true at the end of the input file. In QBasic, EOF(0) returns true after the next record is read past the end of the input file. For example, the following program runs in BASICA, but generates an "Input past end of file" error message in QBasic:

```
5  WHILE NOT EOF(0)
10 LINE INPUT ; A$
15 WEND
```

FIELD

When a random-access file is closed in BASICA with a CLOSE or RESET statement, field variables retain the last value assigned to them by a GET statement. In QBasic, all field variables are reset to null strings.

See Also «Converting BASICA Programs» «Unsupported Keywords» «Version Differences»

BSAVE, BLOAD Statements

«Contents» «Index» «Back»

BSAVE copies the contents of an area of memory to a file.
BLOAD loads a file created by BSAVE into memory.

BSAVE filespec$, offset%, length&
BLOAD filespec$[,offset%]

- **filespec$** For BSAVE, a file to which an area of memory (a byte-for-byte memory image) is copied. For BLOAD, a memory-image file created by a previous BSAVE.
- **offset%** For BSAVE, the offset of the starting address of the area of memory being saved. For BLOAD, the offset of

the address where loading starts.

- **length&** The number of bytes to copy (from 0 through 65,535).

- The starting address of the memory area saved or loaded is determined by the offset and the most recent DEF SEG statement.

See Also «DEF SEG» «VARPTR, VARSEG» «Differences from BASICA»

CHDIR, MKDIR, RMDIR, FILES Statements

«Contents» «Index» «Back»

CHDIR changes a drive's default directory. MKDIR creates a subdirectory. MRDIR removes a subdirectory. FILES displays the contents of the current directory or a specified directory.

```
CHDIR pathname$
MKDIR pathname$
RMDIR pathname$
FILES [filespec$]
```

- **pathname$** The path of the new default directory, subdirectory to create, or subdirectory to remove.
- **filespec$** A filename or path (may include a drive and DOS wildcard characters). If you don't specify a filespec$, FILES displays all files in the current directory.

Example:

```
MKDIR "C:\\TEMP\\TEST"
CHDIR "C:\\TEMP"
FILES
RMDIR "TEST"
```

CLOSE Statement

«Contents» «Index» «Back»

Closes one or more open files or devices.

```
CLOSE [[#]filenumber%[,[#]filenumber%]...]
```

- **filenumber%** The number of an open file or device.

- CLOSE with no arguments closes all open files and devices.

```
Example:
    CLS
    INPUT "Enter filename: ", n$
    OPEN n$ FOR OUTPUT AS #1
    PRINT #1, "This is saved to the file."
    CLOSE
    OPEN n$ FOR INPUT AS #1
    INPUT #1, a$
    PRINT "Read from file: "; a$
    CLOSE
```

See Also «END» «OPEN» «RESET» «STOP»

EOF Function

«Contents» «Index» «Back»

Tests for the end of a file. EOF returns true (nonzero) if the end of a file has been reached.

EOF(filenumber%)

- **filenumber%** The number of an open file.

```
Example:
    CLS
    OPEN "TEST.DAT" FOR OUTPUT AS #1
    FOR i% = 1 TO 10
        WRITE #1, i%, 2 * i%, 5 * i%
    NEXT i%
    CLOSE #1
    OPEN "TEST.DAT" FOR INPUT AS #1
    DO
        LINE INPUT #1, a$
        PRINT a$
    LOOP UNTIL (EOF(1))
```

See Also «CLOSE» «LOC» «LOF» «OPEN»

ERDEV, ERDEV$ Functions

«Contents» «Index» «Back»

ERDEV returns an error code from the last device that generated a critical error. ERDEV$ returns the name of the device that generated the error.

ERDEV
ERDEV$

- The low byte of the value returned by ERDEV contains the DOS error code. The high byte contains device-attribute information.

«Example»

See Also «ERR, ERL» «ON ERROR»

FIELD Statement

Allocates space for variables in a random-access file buffer.

```
FIELD [#]filenumber%, fieldwidth% AS stringvariable$
                    [,fieldwidth% AS stringvariable$] ...
```

- **filenumber%** The number of an open file.
- **fieldwidth%** The number of characters in the field.
- **stringvariable$** A variable that identifies the field and contains field data.

- Record variables usually provide a better way to handle record data.

Example:

```
OPEN "FILEDAT.DAT" FOR RANDOM AS #1 LEN = 80
FIELD #1, 30 AS name$, 50 AS address$
```

See Also «GET, PUT» «LSET, RSET» «TYPE» «Differences from BASICA»

FILEATTR Function

Returns information about an open file.

FILEATTR(filenumber%,attribute%)

- **filenumber%** The number of an open file.
- **attribute%** Specifies the type of information to return. When attribute% is 1, FILEATTR returns a value indicating the file's access mode:

Value	Mode
1	Input
2	Output
4	Random
8	Append
32	Binary

When attribute% is 2, FILEATTR returns the DOS file handle.

Example:

```
OPEN "TEST.DAT" FOR BINARY AS #1
PRINT FILEATTR(1, 1)
CLOSE
```

See Also «OPEN»

FREEFILE Function

Returns the next valid unused file number.

FREEFILE

Example:

```
OPEN "TEST.DAT" FOR OUTPUT AS #1
PRINT "Next file number: "; FREEFILE
CLOSE
```

See Also «OPEN»

GET, PUT Statements (File I/O)

GET reads from a file into a random-access buffer or variable.
PUT writes a variable or random-access buffer to a file.

GET [#]filenumber%[,[recordnumber&][,variable]]
PUT [#]filenumber%[,[recordnumber&][,variable]]

- **filenumber%** The number of an open file.
- **recordnumber&** For random-access files, the number of the record to read or write. For binary-mode files, the byte position where reading or writing starts.
- **variable** For GET, a variable used to receive input from the file. For PUT, a variable that contains output to write to the file. The variable is usually a variable of a user-defined data type.

Example:

```
TYPE TestRecord
    Student AS STRING * 20
    Score AS SINGLE
END TYPE
DIM MyClass AS TestRecord
OPEN "FINAL.DAT" FOR RANDOM AS #1 LEN = LEN(MyClass)
MyClass.Student = "MarySa"
MyClass.Score = 99
PUT #1, 1, MyClass
CLOSE #1
OPEN "FINAL.DAT" FOR RANDOM AS #1 LEN = LEN(MyClass)
GET #1, 1, MyClass
PRINT "STUDENT:", MyClass.Student
PRINT "SCORE:", MyClass.Score
CLOSE #1
KILL "FINAL.DAT"
```

See Also «FIELD» «GET, PUT (Graphics)» «LSET, RSET» «MKn$, CVn Functions» «TYPE»

INPUT$ Function

«Contents» «Index» «Back»

Returns a string of characters read from a specified file.

INPUT$(n[,[#]filenumber%])

- **n** The number of characters (bytes) to read.
- **filenumber%** The number of an open file. If filenumber% is omitted, INPUT$ reads from the keyboard.

Example:

```
    OPEN "TEST.DAT" FOR OUTPUT AS #1
    PRINT #1, "The text"
    CLOSE
    OPEN "TEST.DAT" FOR INPUT AS #1
    PRINT INPUT$(3, 1)        'Print first 3 characters.
    CLOSE
```

See Also «INPUT, LINE INPUT»

IOCTL Statement, IOCTL$ Function

IOCTL transmits a control string to a device driver.
IOCTL$ returns current status information from a device driver.

IOCTL [#]filenumber%, string$
IOCTL$([#]filenumber%)

- **filenumber%** The number of an open device.
- **string$** The control string sent to the device.

- IOCTL control strings and the information returned by IOCTL$ depend on the device driver. See your device-driver documentation for information about IOCTL control strings and what is returned by IOCTL$.

LOC Function

Returns the current position within a file.

LOC(filenumber%)

- **filenumber%** The number of an open file or device.

- For binary files, LOC returns the position of the last byte read or written.
- For random-access files, LOC returns the number of the last record read from or written to the file.

- For sequential files, LOC returns the current byte position in the file, divided by 128.

```
Example:
    OPEN "TEST.DAT" FOR RANDOM AS #1
    FOR i% = 1 TO 10
       PUT #1, , i%
    NEXT i%
    SEEK #1, 2
    GET #1, , i%
    PRINT "Data: "; i%; " Current record: "; LOC(1); " Next: "; SEEK(1)
```

See Also «EOF» «SEEK»

LOCK, UNLOCK Statements

«Contents» «Index» «Back»

LOCK limits or prevents access to a file by a network process.
UNLOCK releases the locks imposed by the last LOCK statement.

```
LOCK [#]filenumber% [,{record& | [start&] TO end&}]
UNLOCK [#]filenumber% [,{record& | [start&] TO end&}]
```

- **filenumber%** The number of an open file.
- **record&** For random-access files, the number of a record to lock, relative to the first record in the file. For binary files, the number of a byte to lock, relative to the first byte in the file.
- **start&** and **end&** The numbers of the first and last records or bytes in a range of records or bytes to lock or unlock.

- For sequential files, LOCK and UNLOCK affect the entire file.

```
Example:
    'This example runs only in a network environment.
    OPEN "TEST.DAT" FOR RANDOM AS #1
    FOR i% = 1 TO 10
        PUT #1, , i%
    NEXT i%
    LOCK #1, 2          'Lock record 2.
    GET #1, 2, i%
    UNLOCK #1, 2        'Unlock record 2.
```

LOF Function

Returns the length of a file in bytes.

LOF(filenumber%)

- **filenumber%** The number of an open file.

Example:

```
INPUT "Enter filename: "; f$
OPEN f$ FOR BINARY AS #1
PRINT "File length = "; LOF(1)
CLOSE
```

LSET, RSET Statements

LSET and RSET move data into a random-access file buffer (in preparation for a PUT statement) and left- or right-justify the value of a string variable. LSET also copies the contents of one record variable to another.

LSET stringvariable$=stringexpression$
RSET stringvariable$=stringexpression$
LSET recordvariable1=recordvariable2

- **stringvariable$** Any string variable or a random-access file field defined in a FIELD statement.
- **stringexpression$** For LSET, the left-justified version of stringvariable$. For RSET, the right-justified version of stringvariable$.
- **recordvariable1** **recordvariable2** Record variables of any user-defined data type. Use LSET to assign a record variable of one data type to a different user-defined data type.

Example:

```
OPEN "FILEDAT.DAT" FOR RANDOM AS #1 LEN = 10
FIELD #1, 5 AS Ls1$, 5 AS Rs1$
LSET Ls1$ = "LSET"
RSET Rs1$ = "RSET"
PUT #1, 1
CLOSE #1
OPEN "FILEDAT.DAT" FOR RANDOM AS #1 LEN = 10
```

```
    FIELD #1, 5 AS Ls2$, 5 AS Rs2$
    GET #1, 1
    PRINT "*" + Ls2$ + "*", "*" + Rs2$ + "*"
    CLOSE #1
```

See Also «FIELD» «GET, PUT»

NAME Statement

Renames a file or directory.

NAME oldspec$ AS newspec$

- **oldspec$** and **newspec$** The name of an existing file and the new name for the file. Each name may include a path.

Example:

```
    INPUT "Old Name: "; OldFN$
    INPUT "New Name: "; NewFN$
    NAME OldFN$ AS NewFN$
```

See Also «FILES»

OPEN Statement

Opens a file or device.

OPEN file$ [FOR mode] [ACCESS access] [lock] AS [#]filenumber% [LEN=reclen%]

- **file$** The name of the file or device. The file name may include a drive and path.
- **mode** One of the following file modes: APPEND, BINARY, INPUT, OUTPUT, or RANDOM. See «OPEN Statement File Modes».
- **access** In network environments, specifies whether the file is opened for READ, WRITE, or READ WRITE access. See «OPEN Statement ACCESS Clause».
- **lock** Specifies the file locking in network environments: SHARED, LOCK READ, LOCK WRITE, LOCK READ WRITE.
- **filenumber%** A number in the range 1 through 255 that identifies the

file while it is open.

- **reclen%** For random-access files, the record length (default is 128 bytes). For sequential files, the number of characters buffered (default is 512 bytes).

Example:

```
INPUT "Enter Filename: "; n$
OPEN n$ FOR OUTPUT AS #1
PRINT #1, "This is saved to the file."
CLOSE
OPEN n$ FOR INPUT AS #1
INPUT #1, a$
PRINT "Read from file: "; a$
CLOSE
```

See Also «CLOSE» «FREEFILE» «OPEN COM» «TYPE» «OPEN Statement Alternate Syntax»

OPEN Statement File Modes

«Contents» «Index» «Back»

The APPEND, BINARY, INPUT, OUTPUT, and RANDOM keywords are used in the OPEN statement to specify file modes. INPUT, OUTPUT, and RANDOM are also used in the OPEN COM statement.

- APPEND specifies that the file is to be opened for sequential output and sets the file pointer to the end of the file. A PRINT # or WRITE # statement then extends (appends to) the file.
- BINARY specifies a binary-file mode. In binary mode, you can read or write information to any byte position in the file using GET or PUT statements.
- INPUT specifies that the file is opened for sequential input.
- OUTPUT specifies that the file is opened for sequential output.
- RANDOM specifies that the file is opened in random-access file mode. RANDOM is the default file mode.

See Also «OPEN» «OPEN COM» «PRINT, LPRINT» «WRITE»

OPEN Statement ACCESS Clause

«Contents» «Index» «Back»

Specifies the type of access users have to an opened file.

ACCESS {**READ** | **WRITE** | **READ WRITE**}

- **READ** Opens a file for reading only.
- **WRITE** Opens a file for writing only.
- **READ WRITE** Opens a file for both reading and writing. READ WRITE mode is valid only for random-access and binary-mode files, and files opened for APPEND (sequential access).

See Also «OPEN»

OPEN Statement Alternate Syntax

OPEN mode2$,[#]filenum%,file$[,reclen%]

- **mode2$** A string expression that begins with one of the following characters and specifies the file mode:
  - O Sequential output mode.
  - I Sequential input mode.
  - R Random-access file input/output mode.
  - B Binary-file mode.
  - A Sequential output mode. Sets the file pointer to the end of the file and the record number to the last record of the file. A PRINT # or WRITE # statement extends (appends to) the file.
- **filenum%** A number in the range 1 through 255 that identifies the file while it is open.
- **file$** The name of the file (may include drive and path).
- **reclen%** For random-access files, the record length in bytes. For sequential files, the number of characters buffered.

- QBasic supports this syntax for compatibility with programs written in earlier versions of BASIC.

See Also «OPEN»

RESET Statement

Closes all open files and devices.

RESET

See Also «CLOSE» «END» «OPEN» «STOP»

SEEK Function and Statement

The SEEK function returns the current file position.
The SEEK statement sets the file position for the next read or write.

SEEK(filenumber%)
SEEK [#]filenumber%, position&

- **filenumber%** The number of an open file.
- **position&** The position where the next read or write occurs. For random-access files, a record number. For other files, the byte position relative to the beginning of the file. The first byte is at position 1.

Example:

```
OPEN "TEST.DAT" FOR RANDOM AS #1
FOR i% = 1 TO 10
    PUT #1, , i%
NEXT i%
SEEK #1, 2
GET #1, , i%
PRINT "Data: "; i%; " Current record: "; LOC(1); " Next: "; SEEK(1)
```

See Also «GET, PUT» «OPEN»

WRITE Statement

Writes data to the screen or a sequential file.

WRITE [[#]filenumber%,] expressionlist

- **filenumber%** The number of an open sequential file. If the file number is omitted, WRITE writes to the screen.
- **expressionlist** One or more variables or expressions, separated by

commas, whose values are written to the screen or file.

- WRITE inserts commas between items and quotation marks around strings as they are written. WRITE writes values to a file in a form that can be read by the INPUT statement.

Example:

```
CLS
OPEN "LIST" FOR OUTPUT AS #1
DO
    INPUT "   NAME:       ", Name$
    INPUT "   AGE:        ", Age$
    WRITE #1, Name$, Age$
    INPUT "Add another entry"; R$
LOOP WHILE UCASE$(R$) = "Y"
CLOSE #1
'Print the file to the screen.
OPEN "LIST" FOR INPUT AS #1
CLS
PRINT "Entries in file:": PRINT
DO WHILE NOT EOF(1)
    INPUT #1, Rec1$, Rec2$   'Read entries from file.
    PRINT Rec1$, Rec2$       'Print the entries on the screen.
LOOP
CLOSE #1
KILL "LIST"
```

See Also «INPUT, LINE INPUT» «OPEN» «PRINT, LPRINT»

AS Keyword

«Contents» «Index» «Back»

Performs different actions as part of several statements:

- Specifies a variable type when used in the COMMON, DECLARE, DEF FN, DIM, FUNCTION, REDIM, SHARED, STATIC, and SUB statements.
- Specifies an element type in a user-defined data type when used in the TYPE statement.
- Assigns a file number to a file or device when used in the OPEN statement.
- Specifies a field name when used in the FIELD statement.
- Specifies a new file name when used in the NAME statement.

See Also «COMMON» «DECLARE» «DEF FN» «DIM, REDIM» «FIELD» «FUNCTION» «NAME» «OPEN» «SHARED, STATIC» «SUB» «TYPE»

BEEP Statement

Generates a beep sound from your computer's speaker.

BEEP

CLS Statement

Clears the screen.

CLS [{0 | 1 | 2}]

CLS	Clears either the text or graphics viewport. If a graphics viewport has been set (using VIEW), clears only the graphics viewport. Otherwise, clears the text viewport or entire screen.
CLS 0	Clears the screen of all text and graphics.
CLS 1	Clears the graphics viewport or the entire screen if no graphics viewport has been set.
CLS 2	Clears the text viewport.

See Also «VIEW» «VIEW PRINT» «WINDOW»

INKEY$ Function

Reads a character from the keyboard.

INKEY$

- INKEY$ returns a null string if there is no character to return.
- For standard keys, INKEY$ returns a 1-byte string containing the character read.
- For extended keys, INKEY$ returns a 2-byte string made up of the null character (ASCII 0) and the keyboard scan code.

```
Example:
    PRINT "Press Esc to exit..."
    DO
    LOOP UNTIL INKEY$ = CHR$(27)    '27 is the ASCII code for Esc.
```

See Also «Keyboard Scan Codes»

INP Function, OUT Statement

INP returns a byte read from a hardware I/O port.
OUT sends a byte to a hardware I/O port.

INP(port%)
OUT port%, data%

- **port%** A number in the range 0 through 65,535 that identifies the port.
- **data%** A numeric expression in the range 0 through 255 to send to the port.

```
Example:
    x% = INP(&H3FC)          'Read COM1 Modem Control Register.
    OUT &H3FC, (x% XOR 1)    'Change Data Terminal Ready bit.
```

See Also «WAIT»

INPUT, LINE INPUT Statements

INPUT reads input from the keyboard or a file. LINE INPUT reads a line of up to 255 characters from the keyboard or a file.

INPUT [;] ["prompt"{; | ,}] variablelist
LINE INPUT [;] ["prompt";] variable$
INPUT #filenumber%, variablelist
LINE INPUT #filenumber%, variable$

- **prompt** An optional literal string that is displayed before the user enters data. A semicolon after prompt appends a question mark to the prompt string.
- **variablelist** One or more variables, separated by commas, in which

data entered from the keyboard or read from a file is stored. Variable names can consist of up to 40 characters and must begin with a letter. Valid characters are A-Z, 0-9, and period (.).

- **variable$** Holds a line of characters entered from the keyboard or read from a file.
- **filenumber%** The number of an open file.

- INPUT uses a comma as a separator between entries. LINE INPUT reads all characters up to a carriage return.
- For keyboard input, a semicolon immediately after INPUT keeps the cursor on the same line after the user presses the Enter key.

Example:

```
CLS
OPEN "LIST" FOR OUTPUT AS #1
DO
    INPUT "   NAME:       ", Name$  'Read entries from the keyboard.
    INPUT "   AGE:        ", Age$
    WRITE #1, Name$, Age$
    INPUT "Add another entry"; R$
LOOP WHILE UCASE$(R$) = "Y"
CLOSE #1
'Echo the file back.
OPEN "LIST" FOR INPUT AS #1
CLS
PRINT "Entries in file:": PRINT
DO WHILE NOT EOF(1)
    LINE INPUT #1, REC$  'Read entries from the file.
    PRINT REC$           'Print the entries on the screen.
LOOP
CLOSE #1
KILL "LIST"
```

See Also «INKEY$» «INPUT$» «OPEN Statement File Modes»

KEY Statement (Assignment)

«Contents» «Index» «Back»

Assigns string values to function keys and, optionally, displays key values.

```
KEY key%, stringexpression$
KEY LIST
KEY ON
KEY OFF
```

- **key%** The number of a function key. Use 1 through 10 for function keys F1 through F10. Use 30 and 31 for function keys F11 and F12 on extended keyboards.

- **stringexpression$** A string of up to 15 characters that is returned when the function key is pressed.
- **LIST** Displays the assignments for each key.
- **ON** Turns on the function-key display line.
- **OFF** Turns off the function-key display line.

```
Example:
    KEY 4, "MENU" + CHR$ (13)
    KEY LIST
    KEY 4, ""
    KEY LIST
```

See Also «KEY, ON KEY (Event Trapping)»

KILL Statement

Deletes files from disk.

KILL filespec$

- **filespec$** Identifies the file or files to delete. It may include a path and the DOS wildcard characters ? and *.

```
Example:
    INPUT "File to delete: "; f$
    KILL f$
```

See Also «FILES»

LOCATE Statement, CSRLIN, POS Functions

LOCATE moves the cursor to a specified position on the screen.
CSRLIN returns the current row position of the cursor.
POS returns the current column position of the cursor.

LOCATE [row%] [,[column%] [,[cursor%] [,start% [,stop%]]]]
CSRLIN
POS(expression)

- **row%** and **column%** The number of the row and column to which the cursor moves.
- **cursor%** Specifies whether the cursor is visible: 0 = invisible, 1 = visible
- **start%** and **stop%** Integer expressions in the range 0 through 31 that specify the first and last cursor scan lines. You can change the cursor size by changing the cursor scan lines.
- **expression** Any expression.

Example:

```
CLS
LOCATE 5, 5
MyRow% = CSRLIN
MyCol% = POS(0)
PRINT "Position 1 (Press any key)"
DO
LOOP WHILE INKEY$ = ""
LOCATE (MyRow% + 2), (MyCol% + 2)
PRINT "Position 2"
```

LPOS Function

Returns the number of characters sent to a printer since the last carriage return was sent.

LPOS(n%)

- **n%** Indicates one of the printer ports: 0 = LPT1, 1 = LPT1, 2 = LPT2, 3 = LPT3

Example:

```
'This example requires a printer.
LPRINT
FOR i% = 1 TO 20
    LPRINT i%;
    IF LPOS(1) >= 10 THEN LPRINT     'Begin a new line.
NEXT i%
```

OPEN COM Statement

Opens and initializes a communications channel for input or output (I/O).

The OPEN COM statement must be executed before a device can be used for communication using an RS232 interface.

OPEN "COMn: optlist1 optlist2" [FOR mode] AS [#]filenum% [LEN=reclen%]

- **n** The communications port to open (1 = COM1, 2 = COM2).
- **optlist1** The most-often-used communications parameters:
  [baud] [,[parity] [,[data] [,[stop]]]]
  baud is the baud rate of the device to be opened:
  75, 110, 150, 300, 600, 1200, 2400, 4800, 9600
  parity is the method of parity checking:
  N (none) E (even) O (odd)
  S (space) M (mark) PE (enable error checking)
  data is the number of data bits per byte:
  5, 6, 7, 8
  stop is the number of stop bits:
  1, 1.5, 2
  Defaults: 300 baud, even parity, 7 data bits, 1 stop bit.
- **optlist2** A list of less-often-used parameters, separated by commas:

Option	Description
ASC	Opens the device in ASCII mode.
BIN	Opens the device in binary mode.
CD[m]	Sets the timeout period (in milliseconds) on the Data Carrier Detect (DCD) line.
CS[m]	Sets the timeout period (in milliseconds) on the Clear to Send (CTS) line.
DS[m]	Sets the timeout period (in milliseconds) on the Data Set Ready (DS) line.
LF	Sends a line-feed character after a carriage return.
OP[m]	Specifies how long (in milliseconds) OPEN COM waits for all communications lines to become open.
RB[n]	Sets the size (in bytes) of the receive buffer.
RS	Suppresses detection of Request to Send (RTS).
TB[n]	Sets the size (in bytes) of the transmit buffer.

- **mode** INPUT, OUTPUT, or RANDOM (the default).
  See «OPEN Statement File Modes».
- **filenum%** A number in the range 1 through 255 that identifies the communications channel as long as it is open.
- **reclen%** Random-access-mode buffer size (default is 128 bytes).

Example:

```
'Use this example for trouble shooting serial communications problems.
'Slow baud, hardware handshaking is ignored and buffers are enlarged.
OPEN "COM1:300,N,8,1,CD0,CS0,DS0,OP0,RS,TB2048,RB2048" FOR RANDOM AS #1
```

See Also «OPEN»

PEN Function

Returns the status of the light pen.

PEN(n%)

- **n%** Specifies the information to be returned about light pen status:

n%	Returns
0	Whether pen was used since last call (-1 if yes, 0 if no)
1	The x screen coordinate of the last pen press
2	The y screen coordinate of the last pen press
3	The current pen switch status (-1 if down, 0 if up)
4	The x screen coordinate where the pen last left the screen
5	The y screen coordinate where the pen last left the screen
6	The character row of the last pen press
7	The character column of the last pen press
8	The character row where the pen last left the screen
9	The character column where the pen last left the screen

Example:

```
DO
  P = PEN(3)
  LOCATE 1, 1: PRINT "Pen is ";
  IF P THEN PRINT "down" ELSE PRINT "up  "
  PRINT "X ="; PEN(4), " Y ="; PEN(5); "  "
LOOP
```

See Also «PEN, ON PEN Statements» «SCREEN» «Screen Modes»

PLAY Statement (Music)

Plays musical notes.

PLAY commandstring$

- **commandstring$** A string expression that contains one or more of the following PLAY commands:

```
Octave and tone commands:
  Ooctave     Sets the current octave (0 - 6).
  < or >      Moves up or down one octave.
  A - G       Plays the specified note in the current octave.
  Nnote       Plays a specified note (0 - 84) in the seven-octave
              range (0 is a rest).


Duration and tempo commands:
  Llength     Sets the length of each note (1 - 64). L1 is whole note,
              L2 is a half note, etc.
  ML          Sets music legato.
  MN          Sets music normal.
  MS          Sets music staccato.
  Ppause      Specifies a pause (1 - 64). P1 is a whole-note pause,
              P2 is a half-note pause, etc.
  Ttempo      Sets the tempo in quarter notes per minute (32 - 255).


Mode commands:
  MF           Plays music in foreground.
  MB           Plays music in background.


Suffix commands:
  # or +       Turns preceding note into a sharp.
  -            Turns preceding note into a flat.
  .            Plays the preceding note 3/2 as long as specified.
```

- To execute a PLAY command substring from a PLAY command string, use the "X" command:

```
PLAY "X"+ VARPTR$(commandstring$)
```

Example:

```
'Play scale in 7 different octaves
scale$ = "CDEFGAB"
PLAY "L16"
FOR i% = 0 TO 6
    PLAY "O" + STR$(i%)
    PLAY "X" + VARPTR$(scale$)
NEXT i%
```

See Also «BEEP» «PLAY Function» «PLAY, ON PLAY (Event Trapping)» «SOUND» «VARPTR$» «Differences from BASICA»

PMAP Function

«Contents» «Index» «Back»

Returns the window coordinate equivalent to a viewport coordinate, as defined by the WINDOW statement, or vice versa.

PMAP (startcoordinate#, n%)

- **startcoordinate#** A window or viewport coordinate.
- **n%** A value indicating which coordinate is returned:

startcoordinate#	n%	Returns
Window x coordinate	0	Viewport x coordinate
Window y coordinate	1	Viewport y coordinate
Viewport x coordinate	2	Window x coordinate
Viewport y coordinate	3	Window y coordinate

Example:

```
'This example requires a graphics adapter that supports screen mode 1.
SCREEN 1
WINDOW SCREEN (0, 0)-(100, 100)
PRINT "Logical x=50, physical x="; PMAP(50, 0)
PRINT "Logical y=50, physical y="; PMAP(50, 1)
```

See Also «POINT» «VIEW» «WINDOW»

POINT Function

«Contents» «Index» «Back»

Returns the current graphics cursor coordinates or the color attribute of a specified pixel.

POINT {(n%) | (x%,y%)}

- **(n%)** Indicates the type of coordinate to return:

n%	Returns
0	The current viewport x coordinate
1	The current viewport y coordinate
2	The current window x coordinate
3	The current window y coordinate

- **(x%,y%)** The coordinates of the pixel that POINT checks for color. If the coordinates are outside the current viewport, POINT returns -1.

Example:

```
    'This example requires a color graphics adapter.
    SCREEN 1
    LINE (0, 0)-(100, 100), 2
    LOCATE 14, 1
    FOR y% = 1 TO 10
        FOR x% = 1 TO 10
            PRINT POINT(x%, y%);
        NEXT x%
        PRINT
    NEXT y%
```

See Also «COLOR» «PMAP» «SCREEN» «VIEW» «WINDOW»
«Color Attributes and Values»

PRESET, PSET Statements

Draw a specified point on the screen.

PRESET [STEP] (x!,y!) [,color%]
PSET [STEP] (x!,y!) [,color%]

- **STEP** Specifies that the x! and y! are expressed relative to the current graphics cursor location.
- **(x!,y!)** The screen coordinates of the pixel to be set.
- **color%** A color attribute that sets the pixel color. If color% is omitted, PRESET uses the current background and PSET uses the current foreground color.

- Available color attributes depend on your graphics adapter and screen mode. Coordinate values depend on the graphics adapter, screen mode, and most recent VIEW and WINDOW statements.

Example:

```
    'This example requires a color graphics adapter.
    SCREEN 1
    FOR i% = 0 TO 320
        PSET (i%, 100)
        FOR delay% = 1 TO 100: NEXT delay%
        PRESET (i%, 100)
    NEXT i%
```

See Also «SCREEN» «VIEW» «WINDOW»
«Color Attributes and Values» «Screen Modes»

PRINT, LPRINT Statements

PRINT writes data to the screen or to a file.
LPRINT prints data on the printer LPT1.

PRINT [#filenumber%,] [expressionlist] [{; | ,}]
LPRINT [expressionlist] [{; | ,}]

- **filenumber%** The number of an open file. If you don't specify a file number, PRINT writes to the screen.
- **expressionlist** A list of one or more numeric or string expressions to print.
- **{; | ,}** Determines where the next output begins:
  ; means print immediately after the last value.
  , means print at the start of the next print zone.
  Print zones are 14 characters wide.

Example:

```
OPEN "TEST.DAT" FOR OUTPUT AS #1
PRINT #1, USING "##.###  "; 12.12345
CLOSE
OPEN "TEST.DAT" FOR INPUT AS #1
INPUT #1, a$
PRINT a$
LPRINT "This is a line"; 1
LPRINT "This is a line",
LPRINT 2
```

See Also «PRINT USING, LPRINT USING» «WIDTH» «WRITE»

PRINT USING, LPRINT USING Statements

PRINT USING writes formatted output to the screen or to a file.
LPRINT USING prints formatted output on the printer LPT1.

PRINT [#filenumber%,] USING formatstring$; expressionlist [{; | ,}]
LPRINT USING formatstring$; expressionlist [{; | ,}]

- **filenumber%** The number of an open sequential file.
- **formatstring$;** A string expression containing one or more «format specifiers».

- **expressionlist** A list of one or more numeric or string expressions to print, separated by commas, semicolons, spaces, or tabs.
- **{; | ,}** Determines where the next output begins:
  ; means print immediately after the last value.
  , means print at the start of the next print zone. Print zones are 14 characters wide.

Example:

```
a = 123.4567
PRINT USING "###.##"; a
LPRINT USING "+###.####"; a
a$ = "ABCDEFG"
PRINT USING "!"; a$
LPRINT USING "\\ \\"; a$
```

See Also «PRINT, LPRINT» «WIDTH»

Format Specifiers

Characters that format a numeric expression

#	Digit position.	+ -	Placed after digit, prints
.	Decimal point position.	+	trailing sign for negative
,	Placed left of the decimal point, prints a comma every third digit.	+	numbers.
		+ $$	Prints leading $.
+	Position of number sign.	+ **	Fills leading spaces with *.
^^^^	Prints in exponential format.	+ **$	Combines ** and $$.

Characters used to format a string expression

&	Prints entire string.	+ \\ \\	Prints first n characters,
!	Prints only the first character of the string.	+	where n is the number of blanks between slashes + 2.

Characters used to print literal characters

_	Prints the following formatting character as a literal.	+	Any character not in this table is printed as a literal.

SOUND Statement

Generates a sound through your computer's speaker.

SOUND frequency, duration

- **frequency** The frequency of the sound in hertz; a value in the range 37 through 32,767.
- **duration** The number of system clock ticks the sound lasts; a value in the range 0 through 65,535. There are 18.2 clock ticks per second.

Example:

```
FOR i% = 440 TO 1000 STEP 5
SOUND i%, i% / 1000
NEXT i%
```

See Also «PLAY»

SPC Function

Skips a specified number of spaces in a PRINT or LPRINT statement.

SPC(n%)

- **n%** The number of spaces to skip; a value in the range 0 through 32,767.

Example:

```
PRINT "Text1"; SPC(10); "Text2"
```

See Also «PRINT, LPRINT» «PRINT USING, LPRINT USING» «SPACE$» «TAB»

STICK Function

Returns the coordinates of a joystick.

STICK(n%)

- **n%** Indicates the coordinate returned:

n%	Returns
0	x coordinate of joystick A
1	y coordinate of joystick A
2	x coordinate of joystick B
3	y coordinate of joystick B

- You must call STICK(0) before STICK(1), STICK (2), or STICK(3). STICK(0) records the current coordinates.

Example:

```
Temp% = STICK(0)
PRINT STICK(2), STICK(3)
```

See Also «STRIG Function» «STRIG, ON STRIG Statements»

TAB Function

Moves the text cursor to a specified print position.

TAB(column%)

- **column%** The column number of the new print position.

Example:

```
PRINT TAB(25); "Text"
```

See Also «PRINT, LPRINT» «PRINT USING, LPRINT USING»
«SPC» «SPACE$»

VIEW PRINT Statement

Sets the boundaries of the screen text viewport.

VIEW PRINT [toprow% TO bottomrow%]

- **toprow%** The number of the top row of the text viewport.
- **bottomrow%** The number of the bottom row of the text viewport.

- If you omit the toprow% and bottomrow% arguments, VIEW PRINT sets the entire screen as the text viewport.
- Ranges for toprow% and bottomrow% depend on the screen mode.

Example:

```
VIEW PRINT 10 TO 15
FOR i% = 1 TO 100      'Output will scroll.
    PRINT i%
NEXT i%
```

See Also «CLS» «LOCATE» «PRINT, LPRINT» «SCREEN» «WIDTH» «Screen Modes»

WAIT Statement

«Contents» «Index» «Back»

Suspends program execution until a specified bit pattern is input from an input port.

WAIT portnumber%, AND-expression% [,XOR-expression%]

- **portnumber%** The number of the input port.
- **AND-expression%** An integer expression that WAIT combines with the bit pattern value using an AND operator. When the result is nonzero, WAIT stops monitoring the port.
- **XOR-expression%** Can be used to turn line bits on and off in the bit pattern before the AND operation is applied.

Example:

```
'Reads the interrupt controller port address &H20.
'Press any key to continue.
WAIT &H20, 1
```

See Also «INP, OUT» «Boolean Operators»

WIDTH Statements

«Contents» «Index» «Back»

Assign an output-line width to a device (such as a printer) or file, or change the number of screen-display columns and rows.

```
WIDTH [columns%] [,rows%]
WIDTH {#filenumber% | device$}, columns%
WIDTH LPRINT columns%
```

- **columns%** The desired width in columns. Screen display width must be 40 or 80 columns.
- **rows%** The desired screen-display height in rows. The value can be 25, 30, 43, 50, or 60, depending on your display adapter and screen mode.
- **#filenumber%** The number of an open file or device.
- **device$** The name of a device:
  SCRN:, COM1:, COM2:, LPT1:, LPT2:, LPT3:

Example:

```
OPEN "LPT1:" FOR OUTPUT AS #1
WIDTH #1, 132
```

See Also «PRINT, LPRINT» «SCREEN» «VIEW PRINT»

WINDOW Statement

«Contents» «Index» «Back»

Defines logical dimensions for the current graphics viewport. Use the WINDOW statement to define your own viewport coordinate system.

```
WINDOW [[SCREEN] (x1!,y1!)-(x2!,y2!)]
```

- **SCREEN** Inverts the normal Cartesian direction of the y screen coordinates so that y values increase from the top of the screen to the bottom.
- **(x1!,y1!)** Logical coordinates that map to the upper-left screen coordinates of the viewport.
- **(x2!,y2!)** Logical coordinates that map to the lower-right screen coordinates of the viewport.

- WINDOW with no arguments disables the logical coordinate system.
- Use the VIEW statement to change the size of the viewport.

Example:

```
'This example requires a color graphics adapter.
SCREEN 1
FOR i% = 1 TO 10 STEP 2
```

```
        WINDOW (-160 / i%, -100 / i%)-(160 / i%, 100 / i%)
        CIRCLE (0, 0), 10
    NEXT i%
```

See Also «CLS» «PMAP» «POINT» «SCREEN» «VIEW» «WIDTH»

USING Keyword

«Contents» «Index» «Back»

Specifies formatting for PRINT USING and LPRINT USING statements and the palette assignments for the PALETTE USING statement.

See Also «PALETTE, PALETTE USING» «PRINT USING, LPRINT USING»

CIRCLE Statement

«Contents» «Index» «Back»

Draws a circle or ellipse on the screen.

CIRCLE [STEP] (x!,y!),radius![,[color%] [,[start!] [,[end!] [,aspect!]]]]

- **STEP** Specifies that coordinates are relative to the current graphics cursor position.
- **(x!,y!)** The coordinates for the center of the circle or ellipse.
- **radius!** The radius of the circle or ellipse in the units of the current coordinate system, determined by the most recent SCREEN, VIEW, and WINDOW statements.
- **color%** A color attribute that sets the circle's color. The available color attributes depend on your graphics adapter and the screen mode set by the most recent SCREEN statement.
- **start!** The starting angle for the arc, in radians.
- **end!** The ending angle for the arc, in radians.
- **aspect!** The ratio of the length of the y axis to the length of the x axis, used to draw ellipses.

- To convert from degrees to radians, multiply degrees by (PI / 180).

Example:

```
    'This example requires a color graphics adapter.
    SCREEN 2
    CIRCLE (320, 100), 200
    CIRCLE STEP (0,0), 100
```

See Also «COLOR» «DRAW» «LINE» «SCREEN» «VIEW» «WINDOW»
«Color Attributes and Values» «Screen Modes»

COLOR Statement

«Contents» «Index» «Back»

Sets the screen display colors.

```
COLOR [foreground%] [,[background%] [,border%]]    Screen mode 0 (text only)
COLOR [background%] [,palette%]                    Screen mode 1
COLOR [foreground%]                                Screen modes 4, 12, 13
COLOR [foreground%] [,background&]                 Screen modes 7-10
```

- **foreground%** / **foreground&** — A number that sets the foreground screen color. In screen mode 0, foreground% is a color attribute that sets the text color. In other screen modes, foreground% a color attribute or 4-bit color value (screen mode 4 only) that sets the text and line-drawing color.
- **background%** / **background&** — A number that sets the background screen color. In screen mode 0, background% is a color attribute. In screen mode 1, background% is a 4-bit color value. In screen modes 7-10, background& is a color value.
- **border%** — A color attribute that sets the screen border color.
- **palette%** — A number (0 or 1) that specifies which of two sets of color attributes to use:

palette%	Attribute 1	Attribute 2	Attribute 3
0	Green	Red	Brown
1	Cyan	Magenta	Bright white

- The available color attributes and values depend on your graphics adapter and the screen mode set by the most recent SCREEN statement.
- If your system is equipped with an EGA, VGA, or MCGA adapter, use the PALETTE statement to change the color assignments of color attributes.

Example:

```
'This example requires a color graphics adapter.
SCREEN 7
FOR i% = 0 TO 15
    COLOR i%
    PRINT i%
NEXT i%
```

See Also «DRAW» «PAINT» «PALETTE, PALETTE USING» «SCREEN»
«Color Attributes and Values» «Screen Modes»

Color Attributes and Values

Color attribute	Color monitor: Default color value(a)	Color monitor: Displayed color	Monochrome monitor: Default color value	Monochrome monitor: Displayed color
SCREEN Modes 0, 7, 8, 9(b), 12, and 13				
0	0	Black	0(c)	Off
1	1	Blue		Underlined(d)
2	2	Green	1(c)	On(d)
3	3	Cyan	1(c)	On(d)
4	4	Red	1(c)	On(d)
5	5	Magenta	1(c)	On(d)
6	6	Brown	1(c)	On(d)
7	7	White	1(c)	On(d)
8	8	Gray	0(c)	Off
9	9	Light Blue		High-intensity Underlined
10	10	Light green	2(c)	High-intensity
11	11	Light cyan	2(c)	High-intensity
12	12	Light red	2(c)	High-intensity
13	13	Light magenta	2(c)	High-intensity
14	14	Yellow	2(c)	High-intensity
15	15	High-intensity white	0(c)	Off
SCREEN Modes 1 and 9(e)				
0	0	Black	0	Off
1	11	Light cyan	2	High-intensity
2	13	Light magenta	2	High-intensity
3	15	High-intensity white	0	Off white
SCREEN Modes 2 and 11				
0	0	Black	0	Off
1	15	High-intensity white	0	Off white

(a) EGA color numbers. VGA and MCGA use display-color values that produce visually equivalent colors.
(b) For VGA or EGA with video memory > 64K.
(c) Only for mode 0.
(d) Off when used for background.
(e) EGA with video memory <= 64K.

See Also «COLOR» «PALETTE, PALETTE USING» «SCREEN» «Screen Modes»

DRAW Statement

Draws an object.

DRAW commandstring$

- **commandstring$** A string expression that contains one or more of the following DRAW commands.

Line-drawing and cursor-movement commands:

D[n%]	Moves cursor down n% units.
E[n%]	Moves cursor up and right n% units.
F[n%]	Moves cursor down and right n% units.
G[n%]	Moves cursor down and left n% units.
H[n%]	Moves cursor up and left n% units.
L[n%]	Moves cursor left n% units.
M[{+|-}]x%,y%	Moves cursor to point x%,y%. If x% is preceded by + or -, moves relative to the current point.
R[n%]	Moves cursor right n% units.
U[n%]	Moves cursor up n% units.
[B]	Optional prefix that moves cursor without drawing.
[N]	Optional prefix that draws and returns cursor to its original position.

Color, rotation, and scale commands:

An%	Rotates an object n% * 90 degrees (n% can be 0, 1, 2, or 3).
Cn%	Sets the drawing color (n% is a color attribute).
Pn1%,n2%	Sets the paint fill and border colors of an object (n1% is the fill-color attribute, n2% is the border-color attribute).
Sn%	Determines the drawing scale by setting the length of a unit of cursor movement. The default n% is 4, which is equivalent to 1 pixel.
TAn%	Turns an angle n% degrees (-360 through 360).

- If you omit n% from line-drawing and cursor-movement commands, the cursor moves 1 unit.
- To execute a DRAW command substring from a DRAW command string, use the "X" command:

```
DRAW "X"+ VARPTR$(commandstring$)
```

Example:

```
'This example requires a color graphics adapter.
SCREEN 1
Triangle$ = "F60 L120 E60"
```

```
    DRAW "C2 X" + VARPTR$(Triangle$)
    DRAW "BD30 P1,2 C3 M-30,-30"
```

See Also «PALETTE, PALETTE USING» «SCREEN» «VARPTR$»
«Color Attributes and Values» «Differences from BASICA»

GET, PUT Statements (Graphics)

GET captures a graphics screen image. PUT displays an image captured by GET.

GET [STEP](x1!,y1!)-[STEP](x2!,y2!), arrayname[(index%)]
PUT [STEP] (x1!,y1!), arrayname[(index%)] [,actionverb]

- **STEP** Specifies that coordinates are relative to the current graphics cursor position.
- **(x1!,y1!)** The upper-left coordinates of the image captured by GET or of the screen location where PUT displays the image.
- **(x2!,y2!)** The lower-right coordinates of the captured image.
- **arrayname** The name of the array where the image is stored. See «Screen Image Arrays and Compatibility» to determine the required size of the array.
- **index%** The array index at which storage of the image begins.
- **actionverb** A keyword indicating how the image is displayed:

Keyword	Action
AND	Merges stored image with an existing image.
OR	Superimposes stored image on existing image.
PSET	Draws stored image, erasing existing image.
PRESET	Draws stored image in reverse colors, erasing existing image.
XOR	Draws a stored image or erases a previously drawn image while preserving the background, producing animation effects.

- A PUT statement should always be executed in the same screen mode as the GET statement used to capture the image, or a compatible mode. See «Screen Image Arrays and Compatibility».

Example:

```
    'This example requires a color graphics adapter.
    SCREEN 1
    DIM Box%(1 TO 200)
    x1% = 0: x2% = 10: y1% = 0: y2% = 10
    LINE (x1%, y1%)-(x2%, y2%), 2, BF
    GET (x1%, y1%)-(x2%, y2%), Box%
    DO
```

```
        PUT (x1%, y1%), Box%, XOR
        x1% = RND * 300
        y1% = RND * 180
        PUT (x1%, y1%), Box%
    LOOP WHILE INKEY$ = ""
```

See Also «SCREEN» «Screen Modes»

Screen Image Arrays and Compatibility

Use bits-per-pixel-per-plane and planes values to determine the required size of the array that holds a graphics screen image. Bits-per-pixel-per-plane and planes values, along with the horizontal resolution, also determine which screen modes are compatibile:

Screen mode	Bits-per-pixel-per-plane	Planes	Horizontal resolution (in pixels)
1	2	1	320
2, 4, 11	1	1	640
3	1	1	720
7	1	4	320
8, 9(> 64K video memory), 12	1	4	640
9(64K video memory), 10	1	2	640
13	8	1	320

The following formula gives the required size, in bytes, of an array used to hold a captured image:

```
size% = 4 + INT(((PMAP (x2!, 0) - PMAP (x1!, 0) + 1) *
        (bits-per-pixel-per-plane%) + 7) / 8) * planes% *
        (PMAP (y2!, 1) - PMAP (y1!, 1) + 1)
```

GET and PUT operations are compatible in screen modes with the same horizontal resolution and bits-per-pixel-per-plane and planes values. For example, screen modes 2, 4, and 11 are compatible, and screen modes 8 and 12 are compatible.

See Also «SCREEN» «Screen Modes»

LINE Statement

Draws a line or rectangle on the screen.

LINE [[STEP](x1!,y1!)]-[STEP](x2!,y2!) [,[color%] [,[B | BF] [,style%]]]

- **STEP** Specifies that coordinates are relative to the current graphics cursor position.
- **(x1!,y1!), (x2!,y2!)** The screen coordinates of the start of the line and of the end of the line.
- **color%** A color attribute that sets the color of the line or rectangle. The available color attributes depend on your graphics adapter and the screen mode set by the most recent SCREEN statement.
- **B** Draws a rectangle instead of a line.
- **BF** Draws a filled box.
- **style%** A 16-bit value whose bits set whether or not pixels are drawn. Use to draw dashed or dotted lines.

Example:

```
'This example requires a color graphics adapter.
SCREEN 1
LINE (110, 70)-(190, 120), , B
LINE (0, 0)-(320, 200), 3, , &HFF00
```

See Also «CIRCLE» «INPUT, LINE INPUT» «PRESET, PSET» «SCREEN» «Color Attributes and Values» «Screen Modes»

PAINT Statement

«Contents» «Index» «Back»

Fills a graphics area with a specified color or pattern.

PAINT [STEP] (x!,y!)[,[{color% | tile$}] [,[bordercolor%] [,background$]]]

- **STEP** Specifies that coordinates are relative to the current graphics cursor position.
- **(x!,y!)** The screen coordinates where painting begins.
- **color%** A color attribute that sets the fill color.
- **tile$** A fill pattern that is 8 bits wide and up to 64 bytes long, defined as follows:

```
tile$ = CHR$(arg1) + CHR$(arg2) + ... + CHR$(argn%)
```

The arguments to CHR$ are numbers between 0 and 255. Each CHR$(argn%) defines a 1-byte, 8-pixel slice of

the pattern based on the binary form of the number.

- **bordercolor%** A color attribute that specifies the color of the filled area's border. PAINT stops filling an area when it encounters a border of the specified color.
- **background$** A 1-byte, 8-pixel background tile slice. Specifying a background tile slice allows you to paint over an area that has already been painted.

- The available color attributes depend on your graphics adapter and the screen mode set by the most recent SCREEN statement.

Example:

```
'This example requires a color graphics adapter.
SCREEN 1
CIRCLE (106, 100), 75, 1
LINE (138, 35)-(288, 165), 1, B
PAINT (160, 100), 2, 1
```

See Also «ASC, CHR$» «CIRCLE» «DRAW» «LINE» «SCREEN» «Color Attributes and Values» «Screen Modes»

PALETTE, PALETTE USING Statements

«Contents» «Index» «Back»

Change the color assignments of color attributes in the current screen mode. PALETTE and PALETTE USING work only on systems equipped with EGA, VGA, or MCGA adapters.

PALETTE [attribute%,color&]
PALETTE USING arrayname#[(index%)]

- **attribute%** The color attribute to change.
- **color&** A color value to assign to an attribute.
- **arrayname#** An array of color values to assign to the current screen mode's set of attributes. The array must be large enough to assign colors to all the attributes.
- **index%** The index of the first array element to assign to an attribute.

- The available color attributes and values depend on your graphics adapter and the screen mode set by the most recent SCREEN statement.

Example:

```
'This example requires a color graphics adapter.
PALETTE 0, 1
SCREEN 1
FOR i% = 0 TO 3: a%(i%) = i%: NEXT i%
```

```
    LINE (138, 35)-(288, 165), 3, BF
    LINE (20, 10)-(160, 100), 2, BF
    DO
        FOR i% = 0 TO 3
            a%(i%) = (a%(i%) + 1) MOD 16
        NEXT i%
        PALETTE USING a%(0)
    LOOP WHILE INKEY$ = ""
```

See Also «COLOR» «SCREEN» «Color Attributes and Values» «Screen Modes»

PCOPY Statement

Copies one video memory page to another.

PCOPY sourcepage%,destinationpage%

- **sourcepage%** The number of a video memory page to copy.
- **destinationpage%** The number of the video memory page to copy to.

- The value that identifies the video page is determined by the size of video memory and the current screen mode.

Example:

```
    PCOPY 1, 3
```

See Also «SCREEN» «Screen Modes»

SCREEN Statement

Sets the screen mode and other characteristics of your screen.

SCREEN mode% [,[colorswitch%] [,[activepage%] [,visualpage%]]]

- **mode%** Sets the screen mode. See «Screen Modes».
- **colorswitch%** A value (0 or 1) that switches between color and monocolor display (modes 0 and 1 only):

Mode	Value	Action
0	0	Disables color
0	Nonzero	Enables color
1	0	Enables color
1	Nonzero	Disables color

- **activepage%** The screen page that text or graphics output writes to.
- **visualpage%** The screen page that is currently displayed on your screen.

Example:

```
'This example requires a color graphics adapter.
SCREEN 1        '320 x 200 graphics
LINE (110, 70)-(190, 120), , B
LINE (0, 0)-(320, 200), 3, , &HFF00
```

See Also «CIRCLE» «COLOR» «DRAW» «LINE» «PAINT» «SCREEN Function» «VIEW» «WINDOW» «Screen Modes»

Screen Modes

«Contents» «Index» «Back»

The following table summarizes screen modes:

----------MDPA, CGA, Hercules, Olivetti, EGA, VGA, or MCGA Adapters----------

SCREEN 0: Text mode only
- 40 x 25, 40 x 43, 40 x 50, 80 x 25, 80 x 43, or 80 x 50 text format, 8 x 8 character box (8 x 14, 9 x 14, or 9 x 16 with EGA or VGA)
- 16 colors assigned to any of 16 attributes (with CGA or EGA)
- 64 colors assigned to any of 16 attributes (with EGA or VGA)
- Depending on the text resolution and adapter, 8 video memory pages (0-7), 4 pages (0-3), 2 pages (0-1), or 1 page (0)

----------CGA, EGA, VGA, or MCGA Adapters----------

SCREEN 1: 320 x 200 graphics
- 40 x 25 text format, 8 x 8 character box
- 16 background colors and one of two sets of 3 foreground colors assigned using COLOR statement with CGA
- 16 colors assigned to 4 attributes with EGA or VGA
- 1 video memory page (0)

SCREEN 2: 640 x 200 graphics
- 80 x 25 text format, 8 x 8 character box
- 16 colors assigned to 2 attributes with EGA or VGA
- 1 video memory page (0)

----------Hercules, Olivetti, or AT&T Adapters----------

SCREEN 3: Hercules adapter required, monochrome monitor only
- 720 x 348 graphics
- 80 x 25 text format, 9 x 14 character box
- Usually 2 video memory pages (0-1); 1 page (0) if a second color display adapter is installed
- PALETTE statement not supported
- Invoke the Hercules driver MSHERC.COM before using screen mode 3

SCREEN 4:
- Supports Olivetti Personal Computers models M24, M240, M28, M280, M380, M380/C, and M380/T and AT&T Personal Computers 6300 series
- 640 x 400 graphics
- 80 x 25 text format, 8 x 16 character box
- 1 of 16 colors assigned as the foreground color (selected by the COLOR statement); background is fixed at black
- 1 video memory page (0)
- PALETTE statement not supported

----------EGA or VGA Adapters----------

SCREEN 7: 320 x 200 graphics
- 40 x 25 text format, 8 x 8 character box
- Assignment of 16 colors to any of 16 attributes
- If 64K EGA adapter memory, 2 video memory pages (0-1); otherwise, 8 pages (0-7)

SCREEN 8: 640 x 200 graphics
- 80 x 25 text format, 8 x 8 character box
- Assignment of 16 colors to any of 16 attributes
- If 64K EGA adapter memory, 1 video memory page (0); otherwise, 4 pages (0-3)

SCREEN 9: 640 x 350 graphics
- 80 x 25 or 80 x 43 text format, 8 x 14 or 8 x 8 character box
- 16 colors assigned to 4 attributes (64K adapter memory), or 64 colors assigned to 16 attributes (more than 64K adapter memory)
- If 64K EGA adapter memory, 1 video memory page (0); otherwise, 2 pages (0-1)

----------EGA or VGA Adapters, Monochrome Monitor Only----------

SCREEN 10: 640 x 350 graphics, monochrome monitor only
- 80 x 25 or 80 x 43 text format, 8 x 14 or 8 x 8 character box
- Up to 9 pseudocolors assigned to 4 attributes
- 2 video memory pages (0-1), 256K adapter memory required

----------VGA or MCGA Adapters----------

Screen 11 (VGA or MCGA)
- 640 x 480 graphics
- 80 x 30 or 80 x 60 text format, 8 x 16 or 8 x 8 character box
- Assignment of up to 256K colors to 2 attributes
- 1 video memory page (0)

Screen 12 (VGA)
- 640 x 480 graphics
- 80 x 30 or 80 x 60 text format, 8 x 16 or 8 x 8 character box
- Assignment of up to 256K colors to 16 attributes
- 1 video memory page (0)

Screen 13 (VGA or MCGA)
- 320 x 200 graphics
- 40 x 25 text format, 8 x 8 character box
- Assignment of up to 256K colors to 256 attributes
- 1 video memory page (0)

See Also «SCREEN Statement»

SCREEN Function

Returns the ASCII value or color attribute of a character at a specified screen location.

SCREEN (row%,column% [,colorflag%])

- **row%** The row coordinate of a character.
- **column%** The column coordinate of a character.
- **colorflag%** A value (0 or 1) that specifies what is returned.

Value	Returns
0 (or omitted)	The character's ASCII code.
1	The character's color attribute.

Example:

```
CLS
PRINT "Hello"
PRINT "The ASCII value of character at 1,1 is"; SCREEN(1, 1)
```

See Also «POINT» «SCREEN Statement» «ASCII Character Codes» «Color Attributes and Values»

VIEW Statement

Defines the size and location of a viewport where graphics can be displayed on the screen.

VIEW [[SCREEN] (x1!,y1!)-(x2!,y2!) [,[color%] [,border%]]]

- **SCREEN** Specifies that coordinates are relative to the screen rather than the viewport.
- **(x1!,y1!)-(x2!,y2!)** The coordinates of diagonally opposite corners of the viewport.
- **color%** A color attribute that sets the viewport fill

color.

- **border%** A color attribute that sets the viewport border color.

- If all arguments are omitted, the entire screen is the viewport.
- The available color attributes depend on your graphics adapter and the screen mode set by the most recent SCREEN statement.

Example:

```
'This example requires a color graphics adapter.
SCREEN 1
VIEW (10, 10)-(300, 180), , 1
LOCATE 1, 11: PRINT "A big graphics viewport";
VIEW SCREEN (80, 80)-(200, 125), , 1
LOCATE 11, 11: PRINT "A small graphics viewport";
```

See Also «CLS» «SCREEN» «VIEW PRINT» «WINDOW» «Color Attributes and Values» «Screen Modes»

ABS, SGN Functions

«Contents» «Index» «Back»

ABS returns the absolute value of a number.
SGN returns a value indicating the sign of a numeric expression (1 if the expression is positive, 0 if it is zero, or -1 if it is negative).

ABS(numeric-expression)
SGN(numeric-expression)

- **numeric-expression** Any numeric expression.

Example:

```
PRINT ABS(45.5 - 100!)          'Output is:  54.5
PRINT SGN(1), SGN(-1), SGN(0)   'Output is:  1  -1  0
```

ASC, CHR$ Functions

«Contents» «Index» «Back»

ASC returns the ASCII code for the first character in a string expression.
CHR$ returns the character corresponding to a specified ASCII code.

```
ASC(stringexpression$)
CHR$(ascii-code%)
```

- **stringexpression$** Any string expression.
- **ascii-code%** The ASCII code of the desired character.

```
Example:
    PRINT ASC("Q")    'Output is:  81
    PRINT CHR$(65)    'Output is:  A
```

See Also «ASCII Character Codes»

ATN, COS, SIN, TAN Functions

«Contents» «Index» «Back»

ATN returns the arctangent of a specified numeric expression.
COS, SIN, and TAN return the cosine, sine, and tangent of a specified angle.

```
ATN(numeric-expression)
COS(angle)
SIN(angle)
TAN(angle)
```

- **numeric-expression** The ratio between the sides of a right triangle.
- **angle** An angle expressed in radians.

- The ATN function returns an angle in radians.
- To convert from degrees to radians, multiply degrees by (PI / 180).

```
Example:
    CONST PI=3.141592654
    PRINT ATN(TAN(PI/4.0)), PI/4.0    'Output is:  .7853981635  .7853981635
    PRINT (COS(180 * (PI / 180)))     'Output is:  -1
    PRINT (SIN(90 * (PI / 180)))      'Output is:  1
    PRINT (TAN(45 * (PI / 180)))      'Output is:  1.000000000205103
```

CDBL, CSNG Functions

«Contents» «Index» «Back»

CDBL converts a numeric expression to a double-precision value.
CSNG converts a numeric expression to a single-precision value.

CDBL(numeric-expression)
CSNG(numeric-expression)

- **numeric-expression** Any numeric expression.

Example:
```
    PRINT 1 / 3, CDBL(1 / 3)    'Output is:  .3333333  .3333333333333333
    PRINT CSNG(975.3421515#)    'Output is:  975.3422
```

See Also «CINT, CLNG»

CINT, CLNG Functions

«Contents» «Index» «Back»

CINT rounds a numeric expression to an integer.
CLNG rounds a numeric expression to a long (4-byte) integer.

CINT(numeric-expression)
CLNG(numeric-expression)

- **numeric-expression** For CINT, any numeric expression in the range -32,768 through 32,767.
  For CLNG, any numeric expression in the range -2,147,483,648 through 2,147,483,647.

Example:
```
    PRINT CINT(12.49), CINT(12.51)    'Output is:  12  13
    PRINT CLNG(338457.8)              'Output is:  338458
```

See Also «CDBL, CSNG» «FIX, INT»

DATE$ Function and Statement

«Contents» «Index» «Back»

The DATE$ function returns the computer's current system date.
The DATE$ statement sets the current system date on your computer.

DATE$
DATE$ = stringexpression$

- **stringexpression$** The date in one of the following forms: mm-dd-yy, mm-dd-yyyy, mm/dd/yy, mm/dd/yyyy.

- The DATE$ function returns a string in the form mm-dd-yyyy.

Example:

```
PRINT DATE$
DATE$ = "01-01-90"    'Note: The new system date remains in effect until
                      '      you change it again.
PRINT "Date set to "; DATE$
```

See Also «TIME$»

EXP, LOG Functions

EXP returns e raised to a specified power, where e is the base of natural logarithms. LOG returns the natural logarithm of a numeric expression.

EXP(numeric-expression)
LOG(numeric-expression)

- **numeric-expression** For EXP, a number less than or equal to 88.02969. For LOG, any positive numeric expression.

Example:

```
PRINT EXP(0), EXP(1)          'Output is:  1  2.718282
PRINT LOG(1), LOG(EXP(1))     'Output is:  0  1
```

FIX, INT Functions

FIX truncates a floating-point expression to its integer portion.
INT returns the largest integer less than or equal to a numeric expression.

```
FIX(numeric-expression)
INT(numeric-expression)
```

- **numeric-expression** Any numeric expression.

```
Example:
    PRINT FIX(12.49), FIX(12.54)    'Output is:  12  12
    PRINT INT(12.54), INT(-99.4)    'Output is:  12  -100
```

See Also «CINT, CLNG»

RANDOMIZE Statement, RND Function

RANDOMIZE initializes the random-number generator.
RND returns a single-precision random number between 0 and 1.

```
RANDOMIZE [seed%]
RND[(n#)]
```

- **seed%** A number used to initialize the random-number generator. If omitted, RANDOMIZE prompts for it.
- **n#** A value that sets how RND generates the next random number:

n#	RND returns
Less than 0	The same number for any n#
Greater than 0 (or omitted)	The next random number
0	The last number generated

```
Example:
    RANDOMIZE TIMER
    x% = INT(RND * 6) + 1
    y% = INT(RND * 6) + 1
    PRINT "Roll of two dice: die 1 ="; x%; "and die 2 ="; y%
```

SQR Function

Returns the square root of a numeric expression.

SQR(numeric-expression)

- **numeric-expression** A value greater than or equal to zero.

```
Example:
    PRINT SQR(25), SQR(2)    'Output is:  5  1.414214
```

TIME$ Function and Statement

«Contents» «Index» «Back»

The TIME$ function returns the computer's current system time.
The TIME$ statement sets the current system time on your computer.

TIME$
TIME$ = stringexpression$

- **stringexpression$** The time in one of the following forms:
  - hh — Sets the hour; minutes and seconds default to 00.
  - hh:mm — Sets the hour and minutes; seconds default to 00.
  - hh:mm:ss — Sets the hour, minutes, and seconds.

- The TIME$ function returns a string in the form hh:mm:ss.

```
Example:
    PRINT TIME$
    TIME$ = "08:00:58"    'Note: The new system time remains in effect until
                          '      you change it again.
    PRINT "Time set to "; TIME$
```

See Also «DATE$»

MOD Arithmetic Operator

«Contents» «Index» «Back»

Divides one number by another and returns the remainder.

numeric-expression1 MOD numeric-expression2

- **numeric-expression1**, **numeric-expression2** — Any numeric expressions. Real numbers are rounded to integers.

Example:

```
PRINT 19 MOD 6.7    'QBasic rounds 6.7 to 7, then divides.
                    'Output is:  5
```

CONST Statement

Declares one or more symbolic constants.

CONST constantname = expression [,constantname = expression]...

- **constantname** — The name of the constant. This name can consist of up to 40 characters and must begin with a letter. Valid characters are A-Z, 0-9, and period (.).
- **expression** — An expression that is assigned to the constant. The expression can consist of literals (such as 1.0), other constants, any arithmetic or logical operators except exponentiation (^), or a single literal string.

Example:

```
CONST PI = 3.141593
INPUT "Radius of Circle: "; r
PRINT "Area = "; PI * r ^ 2
```

DECLARE Statement

Declares a FUNCTION or SUB procedure and invokes argument data type checking.

DECLARE {FUNCTION | SUB} name [([parameterlist])]

- **name** — The name of the procedure.
- **parameterlist** — One or more variables that specify parameters to be passed to the procedure when it is called:

  variable[( )] [AS type] [, variable[( )] [AS type]]...

variable	A Basic variable name.
type	The data type of the variable (INTEGER, LONG, SINGLE, DOUBLE, STRING, or a user-defined data type). ANY allows any data type.

- DECLARE is required if you call SUB procedures without CALL. QBasic automatically generates DECLARE statements when you save your program.

Example:
The program REMLINE.BAS illustrates declaring FUNCTION and SUB procedures. To view or run this program, load REMLINE.BAS using the Open command from the File menu.

See Also «CALL» «FUNCTION» «SUB»

DEF FN Statement

«Contents» «Index» «Back»

Defines a function.

```
DEF FNname[(parameterlist)] = expression
DEF FNname[(parameterlist)]
    [statementblock]
  FNname = expression
    [statementblock]
  EXIT DEF]
    [statementblock]
  END DEF
```

- **parameterlist** One or more arguments in the following form:

  variable[( )] [AS type] [, variable[( )] [AS type]]...

variable	A Basic variable name.
type	The data type of the variable (INTEGER, LONG, SINGLE, DOUBLE, STRING, or a user-defined data type).

- **expression** The return value of the function.

- The FUNCTION statement provides a better way to define a function.

See Also «EXIT» «FUNCTION» «SHARED, STATIC»

ENVIRON$ Function, ENVIRON Statement

«Contents» «Index» «Back»

ENVIRON$ returns a DOS environment string.
ENVIRON changes or adds an environment string in the DOS environment table.

```
ENVIRON$ (env-variable$)
ENVIRON$ (n%)
ENVIRON stringexpression$
```

- **env-variable$** The name of a DOS environment variable.
- **n%** Specifies that ENVIRON$ returns the nth string from the environment string table.
- **stringexpression$** The name and setting of a DOS environment variable (such as PATH or PROMPT) in one of the following forms:

  env-variable$=env-string$
  env-variable$ env-string$

- Changes made by the ENVIRON statement are erased when the program ends.

Example:

```
    ENVIRON "PATH=TEST"
    PRINT ENVIRON$("PATH")
```

FUNCTION Statement

«Contents» «Index» «Back»

Defines a FUNCTION procedure.

```
FUNCTION name [(parameterlist)] [STATIC]
    [statementblock]
  name = expression
    [statementblock]
END FUNCTION
```

- **name** The name of the function and the data type it returns,

specified by a data-type suffix (%, &, !, #, or $).

- **parameterlist** One or more variables that specify parameters to be passed to the function when it is called:

  variable[( )] [AS type] [, variable[( )] [AS type]]...

  variable A Basic variable name.
  type The data type of the variable (INTEGER, LONG, SINGLE, DOUBLE, STRING, or a user-defined data type).

- **STATIC** Specifies that the values of the function's local variables are saved between function calls.
- **expression** The return value of the function.

- When you call the function, you can specify that an argument's value will not be changed by the function by enclosing the argument in parentheses.

Example:
The program REMLINE.BAS illustrates calling FUNCTION procedures. To view or run this program, load REMLINE.BAS using the Open command from the File menu.

See Also «DECLARE» «DEF FN» «EXIT» «SHARED, STATIC» «SUB»

SHELL Statement

«Contents» «Index» «Back»

Suspends execution of a Basic program to run a DOS command or batch file.

SHELL [commandstring$]

- **commandstring$** The name of a DOS command or batch file.

- Your program resumes when the DOS command or batch file completes.
- If you omit the command string, SHELL invokes a DOS shell and displays the DOS prompt. Use the EXIT command to resume your program.

Example:

```
SHELL "DIR"
```

SUB Statement

Defines a SUB procedure.

```
SUB name[(parameterlist)] [STATIC]
  [statementblock]
END SUB
```

- **name** The name of the SUB procedure, up to 40 characters long, with no data type suffix.
- **parameterlist** One or more variables that specify parameters to be passed to the SUB procedure when it is called:

  variable[( )] [AS type] [, variable[( )] [AS type]]...

  variable A Basic variable name.
  type The data type of the variable (INTEGER, LONG, SINGLE, DOUBLE, STRING, or a user-defined data type).

- **STATIC** Specifies that the values of the SUB procedure's local variables are saved between function calls.

- When you call the SUB procedure, you can specify that an argument's value will not be changed by the procedure by enclosing the argument in parentheses.

Example:
The program REMLINE.BAS illustrates calling SUB procedures. To view or run this program, load REMLINE.BAS using the Open command from the File menu.

See Also «CALL» «DECLARE» «EXIT» «FUNCTION» «SHARED, STATIC»

COM, ON COM Statements

COM enables, disables, or suspends event trapping on a communications port. If event trapping is enabled, ON COM branches to a subroutine whenever characters are received at the port.

```
COM(n%) ON
COM(n%) OFF
COM(n%) STOP
ON COM(n%) GOSUB line
```

- **n%** The number of a COM (serial) port (1 or 2).
- **COM(n%) ON** Enables trapping of a communications event.
- **COM(n%) OFF** Disables communications event trapping.
- **COM(n%) STOP** Suspends communications event trapping. Events are processed once event trapping is enabled by COM ON.
- **line** The label or number of the first line of the event-trapping subroutine.

Example:

```
COM(1) ON       'Enable event trapping on port 1.
ON COM(1) GOSUB ComHandler
DO : LOOP WHILE INKEY$ = ""
COM(1) OFF
END


ComHandler:
    PRINT "Something was typed at the terminal attached to COM1."
    RETURN
```

See Also «OPEN COM»

KEY, ON KEY Statements (Event Trapping)

«Contents» «Index» «Back»

KEY enables, disables, or suspends event trapping of a key.
If event trapping is enabled, ON KEY branches to a subroutine whenever the key is pressed.

```
KEY(n%) ON
KEY(n%) OFF
KEY(n%) STOP
ON KEY(n%) GOSUB line
```

- **n%** A value that specifies a function key, direction key, or user-defined key:

n%	Key
0	All keys listed here (KEY(0) ON, KEY(0) OFF, and KEY(0) STOP only).
1-10	Function keys F1-F10.

11	Up Arrow key.
12	Left Arrow key.
13	Right Arrow key.
14	Down Arrow key.
15-25	User-defined keys. For more information, see «Declaring User-Defined Keys».
30, 31	Function keys F11 and F12.

- **KEY(n%) ON** Enables event trapping for the specified key.
- **KEY(n%) OFF** Disables key event trapping.
- **KEY(n%) STOP** Suspends key event trapping. Events are processed once event trapping is enabled by KEY ON.
- **line** The label or number of the first line of the event-trapping subroutine.

Example:

```
    'This example requires Caps Lock and Num Lock to be off.
    CONST ESC = 27
    KEY 15, CHR$(&H4) + CHR$(&H1F)              'Set up Ctrl+S as KEY 15.
    ON KEY(15) GOSUB PauseHandler
    KEY(15) ON
    WHILE INKEY$ <> CHR$(ESC)
        PRINT "Press Esc to stop, Ctrl+S to pause."
        PRINT
    WEND
    END


    PauseHandler:
        SLEEP 1
        RETURN
```

See Also «KEY (Assignment)» «Declaring User-Defined Keys»

Declaring User-Defined Keys

To declare a user-defined key, use the following variation of the KEY statement:

KEY n%, CHR$(keyboardflag%) + CHR$(scancode%)

- **n%** A value in the range 15 through 25 that identifies the key.
- **keyboardflag%** One of the following values, or a sum of values, specifying whether the user-defined key is used in combination with the Shift, Ctrl, Alt, NumLock, or Caps Lock keys, or with extended keys:

Value	Key
0	No keyboard flag
1 through 3	Either Shift key
4	Ctrl key
8	Alt key
32	NumLock key
64	Caps Lock key
128	Extended keys on a 101-key keyboard

To specify multiple shift states, add the values together. For example, a value of 12 specifies that the user-defined key is used in combination with the Ctrl and Alt keys.

- **scancode%** The scan code for the key being declared. See «Keyboard Scan Codes».

See Also «KEY (Assignment)» «KEY, ON KEY (Event Trapping)»

PEN, ON PEN Statements

«Contents» «Index» «Back»

PEN enables, disables, or suspends light-pen event trapping. If event trapping is enabled, ON PEN branches to a subroutine whenever the light pen is activated.

PEN ON
PEN OFF
PEN STOP
ON PEN GOSUB line

- **PEN ON** Enables light-pen event trapping.
- **PEN OFF** Disables light-pen event trapping.
- **PEN STOP** Suspends light-pen event trapping. Events are processed once event trapping is enabled by PEN ON.
- **line** The label or number of the first line of the event-trapping subroutine.

Example:

```
'This example requires a light pen.
ON PEN GOSUB Handler
PEN ON
PRINT "Press Esc to exit."
DO UNTIL INKEY$ = CHR$(27): LOOP
END
```

```
Handler:
    PRINT "Pen is at row"; PEN(6); ", column"; PEN(7)
    RETURN
```

See Also «PEN Function»

PLAY Function

Returns the number of notes in the background music queue.

PLAY(n)

- **n** Any numeric expression.

```
Example:
    Music$ = "MBT180o2P2P8L8GGGL2E-P24P8L8FFFL2D"
    PLAY Music$
    WHILE PLAY(0) > 5: WEND
    PRINT "Just about done!"
```

See Also «PLAY (Music)» «PLAY, ON PLAY (Event Trapping)»

PLAY, ON PLAY Statements (Event Trapping)

PLAY enables, disables, or suspends play event trapping.
If event trapping is enabled, ON PLAY branches to a subroutine whenever the music buffer contains fewer than a specified number of notes.

PLAY ON
PLAY OFF
PLAY STOP
ON PLAY(queuelimit%) GOSUB line

- **PLAY ON** Enables play event trapping.
- **PLAY OFF** Disables play event trapping.
- **PLAY STOP** Suspends play event trapping. Events are processed once event trapping is enabled by PLAY ON.
- **queuelimit%** A number in the range 1 through 32. ON PLAY branches to

a subroutine when there are fewer than queuelimit% notes in the music buffer.

- **line** The label or number of the first line of the event-trapping subroutine.

Example:

```
ON PLAY(3) GOSUB Background
PLAY ON
Music$ = "MBo3L8ED+ED+Eo2Bo3DCL2o2A"
PLAY Music$
LOCATE 2, 1: PRINT "Press any key to stop.";
DO WHILE INKEY$ = "": LOOP
END


Background:
    i% = i% + 1
    LOCATE 1, 1: PRINT "Background called "; i%; "time(s)";
    PLAY Music$
    RETURN
```

See Also «PLAY (Music)» «PLAY Function»

STRIG Function

«Contents» «Index» «Back»

Returns the status of a joystick trigger.

STRIG(n%)

- **n%** A value that specifies a joystick status condition:

n%	Condition
0	Lower joystick A trigger was pressed since last STRIG(0)
1	Lower joystick A trigger is currently pressed
2	Lower joystick B trigger was pressed since last STRIG(2)
3	Lower joystick B trigger is currently pressed
4	Upper joystick A trigger was pressed since last STRIG(4)
5	Upper joystick A trigger is currently pressed
6	Upper joystick B trigger was pressed since last STRIG(6)
7	Upper joystick B trigger is currently pressed

- STRIG returns -1 if the condition is true, 0 otherwise.

Example:

```
PRINT "Press Esc to exit."
```

```
    DO
        IF STRIG(0) OR INKEY$ = CHR$(27) THEN EXIT DO
    LOOP
    DO
        BEEP                    'BEEP while trigger A is pressed.
    LOOP WHILE STRIG(1)
```

See Also «STICK» «STRIG, ON STRIG Statements»

STRIG, ON STRIG Statements

«Contents» «Index» «Back»

STRIG enables, disables, or suspends joystick event trapping.
If event trapping is enabled, ON STRIG branches to a subroutine whenever a specified joystick trigger is pressed.

STRIG(n%) ON
STRIG(n%) OFF
STRIG(n%) STOP
ON STRIG(n%) GOSUB line

- **n%** A value that specifies a joystick trigger:

n%	Trigger
0	Lower trigger, joystick A
2	Lower trigger, joystick B
4	Upper trigger, joystick A
6	Upper trigger, joystick B

- **STRIG(n%) ON** Enables joystick event trapping.
- **STRIG(n%) OFF** Disables joystick event trapping.
- **STRIG(n%) STOP** Suspends joystick event trapping. Events are processed once event trapping is enabled by STRIG ON.
- **line** The label or number of the first line of the event-trapping subroutine.

Example:

```
    'This example requires a joystick.
    ON STRIG(0) GOSUB Handler
    STRIG(0) ON
    PRINT "Press Esc to exit."
    DO UNTIL INKEY$ = CHR$(27): LOOP
    END

Handler:
```

```
    PRINT "Joystick trigger is depressed."
    RETURN
```

See Also «STICK» «STRIG Function»

TIMER Function

Returns the number of seconds elapsed since midnight.

TIMER

- Use TIMER to time programs or parts of programs, or with the RANDOMIZE statement to seed the random-number generator.

Example:

```
    RANDOMIZE TIMER
```

See Also «RANDOMIZE, RND» «TIMER, ON TIMER Statements»

TIMER, ON TIMER Statements

TIMER enables, disables, or suspends timer event trapping.
If event trapping is enabled, ON TIMER branches to a subroutine whenever a specified number of seconds has elapsed.

TIMER ON
TIMER OFF
TIMER STOP
ON TIMER(n%) GOSUB line

- **TIMER ON** Enables timer event trapping.
- **TIMER OFF** Disables timer event trapping.
- **TIMER STOP** Suspends timer event trapping. Events are processed once event trapping is enabled by TIMER ON.
- **n%** The number of seconds that elapse before ON TIMER branches to the event-trapping subroutine; a value in the range 1 through 86,400 (24 hours).
- **line** The label or number of the first line of the event-trapping subroutine.

Example:

```
    ON TIMER(1) GOSUB TimeUpdate
    TIMER ON
    CLS
    PRINT "Time: "; TIME$
    StartTime = TIMER
    WHILE TimePast < 10
        TimePast = TIMER - StartTime
    WEND
    END


    TimeUpdate:
        LOCATE 1, 7: PRINT TIME$
        RETURN
```

See Also «TIMER Function»

ON Keyword

«Contents» «Index» «Back»

Performs different actions as part of several statements:

- Enables error trapping when used with the ON ERROR statement.
- Enables event trapping when used with the event ON statements (COM ON, KEY ON, PEN ON, PLAY ON, STRIG ON, and TIMER ON).
- Specifies an event to trap when used with the ON event statements (ON COM, ON KEY, ON PEN, ON PLAY, ON STRING, and ON TIMER).
- Specifies an expression to evaluate when used with the ON...GOSUB and ON...GOTO statements.

See Also «COM, ON COM» «KEY, ON KEY» «ON ERROR»
«ON...GOSUB, ON...GOTO» «PEN, ON PEN»
«PLAY, ON PLAY» «STRIG, ON STRIG» «TIMER, ON TIMER»

OFF Keyword - Different Uses

«Contents» «Index» «Back»

Turns off the display of function key assignments when used with the KEY (Assignment) statement, or disables event trapping when used with the event OFF statements (COM OFF, KEY OFF, PEN OFF, PLAY OFF, STRIG OFF, and TIMER OFF).

See Also «COM, ON COM» «KEY, ON KEY» «KEY (Assignment)»
«PEN, ON PEN» «PLAY, ON PLAY» «STRIG, ON STRIG»
«TIMER, ON TIMER»

$STATIC, $DYNAMIC Metacommands

«Contents» «Index» «Back»

Set the default array storage.

{REM | '} $STATIC
{REM | '} $DYNAMIC

- **{REM | '}** REM or a remark character (') must precede metacommands.
- **$STATIC** Specifies that arrays declared in subsequent DIM statements are static arrays (unless they are declared in a non-static SUB or FUNCTION procedure). Array storage is allocated when you start the program, and remains fixed.
- **$DYNAMIC** Specifies that arrays declared in subsequent DIM statements are dynamic arrays. Array storage is allocated dynamically while the program runs.

- DIM and REDIM usually provide a better way to specify whether arrays are dynamic or static.

See Also «DIM, REDIM» «REM» «SHARED, STATIC»

COMMON Statement

«Contents» «Index» «Back»

Defines global variables that can be shared throughout a program or between chained programs.

COMMON [SHARED] variablelist

- **SHARED** Indicates that variables are shared with all SUB or FUNCTION procedures.
- **variablelist** One or more variables to be shared:

variable[( )] [AS type] [, variable[( )] [AS type]]...

variable	A Basic variable name. Variable names can consist of up to 40 characters and must begin with a letter. Valid characters are A-Z, 0-9, and period (.).
type	The data type of the variable (INTEGER, LONG, SINGLE, DOUBLE, STRING, or a user-defined data type).

- Unless it has been declared as a static array in a preceding DIM statement, an array variable in a COMMON statement is a dynamic array. Its dimensions must be set in a later DIM or REDIM statement.

See Also «CHAIN» «DIM, REDIM» «FUNCTION» «SHARED, STATIC» «SUB»

DATA, READ, RESTORE Statements

«Contents» «Index» «Back»

DATA specifies values to be read by subsequent READ statements.
READ reads those values and assigns them to variables.
RESTORE allows READ to reread values in specified DATA statements.

```
DATA constant[,constant]...
READ variablelist
RESTORE [line]
```

- **constant** One or more numeric or string constants specifying the data to be read. String constants containing commas, colons, or leading or trailing spaces are enclosed in quotation marks (" ").
- **variablelist** One or more variables, separated by commas, that are assigned data values. Variable names can consist of up to 40 characters and must begin with a letter. Valid characters are A-Z, 0-9, and period (.).
- **line** The label or line number of a DATA statement. If line is omitted, the next READ statement reads values in the first DATA statement in the program.

- DATA statements can be entered only at the module level. They cannot be used in procedures.

Example:

```
FOR i% = 1 TO 3
    READ a%, b$
    PRINT a%, b$
    RESTORE
```

```
    NEXT i%
    DATA 1, "Repeat"
```

DEFtype Statements

Sets the default data type for variables, DEF FN functions, and FUNCTION procedures.

```
DEFINT letterrange [,letterrange]...
DEFLNG letterrange [,letterrange]...
DEFSNG letterrange [,letterrange]...
DEFDBL letterrange [,letterrange]...
DEFSTR letterrange [,letterrange]...
```

- **letterrange** A letter or range of letters (such as A-M). QBasic sets the default data type for variables, DEF FN functions, and FUNCTION procedures whose names begin with the specified letter or letters as follows:

Statement	Default Data Type
DEFINT	Integer
DEFLNG	Long integer
DEFSNG	Single precision
DEFDBL	Double precision
DEFSTR	String

- A data-type suffix (%, &, !, #, or $) always takes precedence over a DEFtype statement.
- Single-precision is the default data type if you do not specify a DEFtype statement.
- After you specify a DEFtype statement in your program, QBasic automatically inserts a corresponding DEFtype statement in each procedure you create.

Example:

```
    DEFDBL A-Z
    a = SQR(3)
    PRINT "Square root of 3 = "; a
```

See Also «Differences from BASICA»

DIM, REDIM Statements

DIM declares an array or specifies a data type for a nonarray variable.
REDIM declares or resizes a dynamic array, erasing any previous values.

```
DIM [SHARED] variable[(subscripts)] [AS type]
             [,variable[(subscripts)] [AS type]]...
REDIM [SHARED] variable(subscripts) [AS type]
               [,variable(subscripts) [AS type]]...
```

- **SHARED** Specifies that variables are shared with all SUB or FUNCTION procedures in the module.
- **variable** The name of an array or variable.
- **subscripts** Dimensions of the array, expressed as follows:

  ```
  [lower TO] upper [,[lower TO] upper]...
  ```

  lower The lower bound of the array's subscripts. The default lower bound is zero.
  upper The upper bound.

- **AS type** Declares the data type of the array or variable (INTEGER, LONG, SINGLE, DOUBLE, STRING, or a user-defined data type).

- DIM declares either static or dynamic arrays. Unless array storage has been determined by $STATIC, $DYNAMIC, or COMMON, arrays dimensioned with numbers are static and arrays dimensioned with variables are dynamic. REDIM always declares dynamic arrays.
- Static array storage is allocated when you start a program and remains fixed. Dynamic array storage is allocated while a program runs.

Example:

```
    ' $DYNAMIC
    DIM A(49, 49)
    REDIM A(19, 14)
```

See Also «COMMON» «ERASE» «OPTION BASE» «SHARED, STATIC» «$STATIC, $DYNAMIC» «Differences from BASICA»

ERASE Statement

Reinitializes array elements or frees dynamic array storage space.

ERASE arrayname [,arrayname]...

- **arrayname** The name of an array.

- For static arrays, ERASE sets each element of a numeric array to zero and each element of a string array to null.
- For dynamic arrays, ERASE frees the memory used by the array. You must redeclare the array's dimensions with REDIM or DIM before using it.

Example:

```
DIM a%(0)
a%(0) = 6
PRINT "Before: "; a%(0)
ERASE a%
PRINT "After: "; a%(0)
```

See Also «CLEAR» «DIM, REDIM»

LBOUND, UBOUND Functions

«Contents» «Index» «Back»

Return the lower and upper bound (smallest or largest available subscript) for the specified array dimension.

LBOUND(array[,dimension%])
UBOUND(array[,dimension%])

- **array** The name of the array.
- **dimension%** Indicates the array dimension whose lower or upper bound is returned. Use 1 for the first dimension, 2 for the second dimension, etc. The default is 1.

Example:

```
DIM a%(1 TO 3, 2 TO 7)
PRINT LBOUND(a%, 1), UBOUND(a%, 2)
```

See Also «DIM, REDIM»

LET Statement

Assigns the value of an expression to a variable.

[LET] variable=expression

- **variable** Any variable. Variable names can consist of up to 40 characters and must begin with a letter. Valid characters are A-Z, 0-9, and period (.).
- **expression** Any expression that provides a value to assign.

- Use of the optional LET keyword is not recommended. The variable=expression assignment statement performs the same action with or without LET.

See Also «LSET, RSET»

OPTION BASE Statement

Sets the default lower bound for array subscripts.

OPTION BASE {0 | 1}

- The DIM statement TO clause provides a better way to set the lower bound of an array subscript.

See Also «DIM, REDIM» «LBOUND, UBOUND»

REM Statement

Allows explanatory remarks to be inserted in a program.

REM remark
' remark

- **remark** Any text.

- Remarks are ignored when the program runs unless they contain metacommands.
- A remark can be inserted on a line after an executable statement if it is preceded by the single-quote (') form of REM or if REM is preceded by a colon (:).

Example:

```
REM    This is a comment.
'      This is also a comment.
PRINT "Test1"       'This is a comment after a PRINT statement.
PRINT "Test2" :  REM This is also a comment after a PRINT statement.
```

See Also «$STATIC, $DYNAMIC»

SHARED, STATIC Statements

«Contents» «Index» «Back»

SHARED gives procedures access to module-level variables.
STATIC makes a variable local to a function or procedure and preserves its value between calls.

SHARED variable[()] [AS type] [,variable[()] [AS type]]...
STATIC variable[()] [AS type] [,variable[()] [AS type]]...

- **variable** The name of the module-level variable to share or variable to make static. Variable names can consist of up to 40 characters and must begin with a letter. Valid characters are A-Z, 0-9, and period (.).
- **AS type** Declares the data type of the variable (INTEGER, LONG, SINGLE, DOUBLE, STRING, or a user-defined type).

Example:

The program REMLINE.BAS illustrates using the SHARED and STATIC statements. To view or run this program, load REMLINE.BAS using the Open command from the File menu.

See Also «COMMON» «DIM, REDIM»

SWAP Statement

Exchanges the values of two variables.

SWAP variable1, variable2

- **variable1** and **variable2** Two variables of the same data type.

Example:

```
a% = 1: b% = 2
PRINT "Before: "; a%, b%
SWAP a%, b%
PRINT "After: "; a%, b%
```

TYPE Statement

Defines a data type containing one or more elements.

```
TYPE usertype
  elementname AS typename
  [elementname AS typename]
.
.
.
END TYPE
```

- **usertype** The name of the data type being defined. The name can consist of up to 40 characters and must begin with a letter. Valid characters are A-Z, 0-9, and period (.).
- **elementname** An element of the user-defined data type.
- **typename** The element's type (INTEGER, LONG, SINGLE, DOUBLE, STRING, or a user-defined data type).

- Use DIM, REDIM, COMMON, STATIC, or SHARED to create a variable of a user-defined data type.

Example:

```
TYPE Card
    Suit AS STRING * 9
    Value AS INTEGER
```

```
    END TYPE
    DIM Deck(1 TO 52) AS Card
    Deck(1).Suit = "Club"
    Deck(1).Value = 2
    PRINT Deck(1).Suit, Deck(1).Value
```

See Also «COMMON» «DIM, REDIM» «SHARED, STATIC»

Data Type Keywords

Specify the data type for a variable in a declarative statement or parameter list:

- **INTEGER** A 16-bit signed integer variable.
- **LONG** A 32-bit signed integer variable.
- **SINGLE** A single-precision 32-bit floating-point variable.
- **DOUBLE** A double-precision 64-bit floating-point variable.
- **STRING * n%** A fixed-length string variable n% bytes long.
- **STRING** A variable-length string variable.

See Also «AS» «Basic Character Set» «COMMON» «DECLARE» «DEF FN» «DIM, REDIM» «FUNCTION» «SHARED, STATIC» «SUB» «TYPE»

CALL Statement

Transfers control to a SUB procedure.

[CALL] name [([argumentlist])]

- **name** The name of the SUB procedure to call.
- **argumentlist** The variables or constants to pass to the SUB procedure. Separate multiple arguments with commas. Specify array arguments with the array name followed by empty parentheses.

- If you omit the CALL keyword, also omit the parentheses around argumentlist. Either declare the procedure in a DECLARE statement before calling it, or save the program and QBasic automatically

generates a DECLARE statement.
- To specify an argument whose value will not be changed by the procedure, enclose the argument in parentheses.

Example:
The program REMLINE.BAS illustrates calling SUB procedures. To view or run this program, load REMLINE.BAS using the Open command from the File menu.

See Also «CALL ABSOLUTE» «DECLARE» «SUB»

CALL ABSOLUTE Statement

«Contents» «Index» «Back»

Transfers control to a machine-language procedure.

CALL ABSOLUTE ([argumentlist,] offset%)

- **argumentlist** Arguments passed to a machine-language procedure as offsets from the current data segment.
- **offset%** The offset from the current code segment, set by DEF SEG, to the starting location of the procedure.

Example:
```
'Calls routine for printing the screen to a local printer.
DIM a%(2)
DEF SEG = VARSEG(a%(0))
FOR i% = 0 TO 2
   READ d%
   POKE VARPTR(a%(0)) + i%, d%
NEXT i%
DATA 205, 5, 203  : ' int 5  retf  'Machine-language code
                                   'for printing screen.
CALL ABSOLUTE(VARPTR(a%(0)))
DEF SEG
```

See Also «CALL» «VARPTR, VARSEG» «Differences from BASICA»

CHAIN Statement

«Contents» «Index» «Back»

Transfers control from the current program to another Basic program.

CHAIN filespec$

- **filespec$** The name of the program to which control is passed.

Example:

```
'Assumes the program TEST.BAS is in a \\DOS directory.
CHAIN "C:\\DOS\\TEST.BAS"
```

See Also «CALL» «COMMON» «RUN» «Differences from BASICA»

DO...LOOP Statement

«Contents» «Index» «Back»

Repeats a block of statements while a condition is true or until a condition becomes true.

```
DO [{WHILE | UNTIL} condition]
    [statementblock]
LOOP
```

```
DO
    [statementblock]
LOOP [{WHILE | UNTIL} condition]
```

- **condition** A numeric expression that Basic evaluates as true (nonzero) or false (zero).

Example:

```
i% = 0
PRINT "Value of i% at beginning of loop is  "; i%
DO WHILE i% < 10
    i% = i% + 1
LOOP
PRINT "Value of i% at end of loop is  "; i%
```

See Also «EXIT» «FOR...NEXT» «WHILE...WEND»

END Statement

«Contents» «Index» «Back»

Ends a program, procedure, block, or user-defined data type.

END [{DEF | FUNCTION | IF | SELECT | SUB | TYPE}]

- **DEF** Ends a multiline DEF FN function definition.
- **FUNCTION** Ends a FUNCTION procedure definition.
- **IF** Ends a block IF...THEN...ELSE statement.
- **SELECT** Ends a SELECT CASE block.
- **SUB** Ends a SUB procedure.
- **TYPE** Ends a user-defined data type definition.

- If no argument is supplied, END ends the program and closes all files.

Example:

```
PRINT "Game over."
END
```

See Also «DEF FN» «FUNCTION» «IF...THEN...ELSE» «SELECT CASE» «STOP» «SUB» «SYSTEM» «TYPE»

EXIT Statement

«Contents» «Index» «Back»

Exits a DO or FOR loop, a FUNCTION or SUB procedure, or a DEF FN function.

EXIT {DEF | DO | FOR | FUNCTION | SUB}

- **DEF** Exits a DEF FN function.
- **DO** Exits a DO loop.
- **FOR** Exits a FOR loop.
- **FUNCTION** Exits a FUNCTION procedure.
- **SUB** Exits a SUB procedure.

Example:

```
i% = 0
DO
    i% = i% + 1
    IF i% = 500 THEN EXIT DO
LOOP
PRINT "EXIT at"; i%
```

See Also «DEF FN» «DO...LOOP» «FOR...NEXT» «FUNCTION» «SUB»

FOR...NEXT Statement

Repeats a block of statements a specified number of times.

```
FOR counter = start TO end [STEP increment]
    [statementblock]
NEXT [counter [,counter]...]
```

- **counter** A numeric variable used as the loop counter.
- **start** and **end** The initial and final values of the counter.
- **increment** The amount the counter is changed each time through the loop.

Example:

```
FOR i% = 1 TO 15
    PRINT i%
NEXT i%
FOR i% = 7 to -6 STEP -3
    PRINT i%
NEXT i%
```

See Also «DO...LOOP» «EXIT» «WHILE...WEND»

GOSUB...RETURN Statement

Branches to and returns from a subroutine.

```
GOSUB line1
.
.
.
RETURN [line2]
```

- **line1** The label or line number of the first line of the subroutine.
- **line2** The label or line number where the subroutine returns.

- If you don't supply a label or line number for RETURN, the program continues execution at the statement following the GOSUB (for subroutine calls) or where an event occurred (for event handling). See the «ON» keyword for information about event-handling statements.

- SUB and CALL statements provide a better alternative to GOSUB subroutines.

Example:

```
FOR i% = 1 TO 20
    GOSUB Square
NEXT i%
END


Square:
PRINT i%, i% * i%
RETURN
```

See Also «CALL» «ON Keyword» «ON...GOSUB» «SUB»

GOTO Statement

«Contents» «Index» «Back»

Branches to a specified line.

GOTO line

- **line** The label or number of the line to execute next.

- DO...LOOP, SELECT CASE, IF...THEN...ELSE, SUB, and FUNCTION provide better ways to control the flow of your program.
- GOTO is also used as a keyword in the ON ERROR statement.

«Example»

See Also «DO...LOOP» «FUNCTION» «IF...THEN...ELSE» «ON ERROR» «ON...GOTO» «SELECT CASE» «SUB»

IF...THEN...ELSE Statement

«Contents» «Index» «Back»

Executes a statement or statement block depending on specified conditions.

IF condition1 THEN

```
    [statementblock-1]
[ELSEIF condition2 THEN
    [statementblock-2]]...
[ELSE
    [statementblock-n]]
END IF
```

```
IF condition THEN statements [ELSE statements]
```

- **condition1**, **condition2** — Any expression that can be evaluated as true (nonzero) or false (zero).
- **statementblock-1**, **statementblock-2**, **statementblock-n** — One or more statements on one or more lines.
- **statements** — One or more statements, separated by colons.

Example:

```
    INPUT "1 or 2? ", i%
    IF i% = 1 OR i% = 2 THEN
        PRINT "OK"
    ELSE
        PRINT "Out of range"
    END IF
```

See Also «ON...GOSUB» «ON...GOTO» «SELECT CASE»

ON...GOSUB, ON...GOTO Statements

«Contents» «Index» «Back»

Branch to one of several locations, depending on the value of an expression.

```
ON expression% GOSUB line-list
ON expression% GOTO line-list
```

- **expression%** — An expression in the range 0 through 255.
- **line-list** — A set of labels or line numbers. If the value of the expression is 1, the program branches to the first line in the list; if the expression is 2, it branches to the second line, and so on.

- SELECT CASE provides a better way to perform multiple branching.

Example:

```
    FOR i% = 1 TO 2
        ON i% GOSUB One, Two
```

```
    NEXT i%
    END


    One: PRINT "One"
    RETURN
    Two: PRINT "Two"
    RETURN
```

See Also «ON Keyword» «SELECT CASE»

RUN Statement

Runs the current program or a specified program.

RUN [{linenumber | file$}]

- **linenumber** The line number in the current program where execution should begin. If no line number is specified, execution begins at the first executable line.
- **file$** The name of a Basic source file. QBasic assumes a .BAS extension.

- RUN closes all files and clears program memory before loading a program. Use the CHAIN statement to run a program without closing open files.

Example:

```
    'Assumes the program TEST.BAS is in a \\DOS directory.
    RUN "C:\\DOS\\TEST.BAS"
```

See Also «CHAIN»

SELECT CASE Statement

Executes one of several statement blocks depending on the value of an expression.

SELECT CASE testexpression

```
CASE expressionlist1
  [statementblock-1]
[CASE expressionlist2
  [statementblock-2]]...
[CASE ELSE
  [statementblock-n]]
END SELECT
```

- **testexpression** Any numeric or string expression.
- **expressionlist1** One or more expressions to match testexpression.
  expressionlist2 The IS keyword must precede any relational operators in an expression.
- **statementblock-1** One or more statements on one or more lines.
  statementblock-2
  statementblock-n

- The expressionlist arguments can have any of these forms or a combination of them, separated by commas:

```
expression[,expression]...
expression TO expression
IS relational-operator expression
```

expression	Any numeric or string expression compatible with testexpression.
relational-operator	One of the following relational operators: <, <=, >, >=, <>, or =.

Example:

```
INPUT "Enter acceptable level of risk (1-5): ", Total
SELECT CASE Total

    CASE IS >= 5
        PRINT "Maximum risk and potential return."
        PRINT "Choose stock investment plan."

    CASE 2 TO 4
        PRINT "Moderate to high risk and potential return."
        PRINT "Choose mutual fund or corporate bonds."

    CASE 1
        PRINT "No risk, low return."
        PRINT "Choose IRA."

END SELECT
```

See Also «IF...THEN...ELSE»

SLEEP Statement

Suspends program execution.

SLEEP [seconds&]

- **seconds&** Number of seconds to suspend the program.
- If seconds& is 0 or is omitted, the program is suspended until a key is pressed or a trapped event occurs.

Example:

```
PRINT "Taking a 10-second nap..."
SLEEP 10
PRINT "Wake up!"
```

See Also «WAIT»

STOP Statement

Halts a program.

STOP

- The STOP keyword also suspends trapping of events in these statements:

«COM, ON COM» «KEY, ON KEY» «PEN, ON PEN»
«PLAY, ON PLAY» «STRIG, ON STRIG» «TIMER, ON TIMER»

Example:

```
FOR i% = 1 TO 10
    PRINT i%
    IF i% = 5 THEN STOP     'STOP pauses; F5 Continues.
NEXT i%
```

See Also «END» «SYSTEM»

SYSTEM Statement

Closes all open files and returns control to the operating system.

SYSTEM

See Also «END» «STOP»

TRON, TROFF Statements

TRON and TROFF enable and disable tracing of program statements.

TRON
TROFF

- QBasic's debugging features make these statements unnecessary.

See Also «Run and Debug Keys»

WHILE...WEND Statement

Executes a series of statements as long as a specified condition is true.

WHILE condition
.
.
.
WEND

- **condition** A numeric expression that Basic evaluates as true (nonzero) or false (zero).

- DO...LOOP provides a better way to execute statements in a program loop.

See Also «DO...LOOP» «FOR...NEXT»

Boolean Operators

Boolean operators perform bit manipulations, Boolean operations, or tests on multiple relations. They return a true (nonzero) or false (zero) value to be used in making a decision.

result = expression1 boolean-operator expression2

- **boolean-operator** Any of the following Boolean operators:

NOT	Bit-wise complement
AND	Conjunction
OR	Disjunction (inclusive "or")
XOR	Exclusive "or"
EQV	Equivalence
IMP	Implication

- Each operator returns results as indicated in the following truth table. T is true (nonzero); F is false (zero):

Expression1	Expression2	NOT	AND	OR	XOR	EQV	IMP
T	T	F	T	T	F	T	T
T	F	F	F	T	T	F	F
F	T	T	F	T	T	F	T
F	F	T	F	F	F	T	T

- Boolean operations are performed after arithmetic and relational operations in order of precedence.
- Expressions are converted to integers or long integers before a Boolean operation is performed.
- If the expressions evaluate to 0 or -1, a Boolean operation returns 0 or -1 as the result. Because Boolean operators do bit-wise calculations, using values other than 0 for false and -1 for true may produce unexpected results.

NEXT Keyword

Increments and tests the counter in a FOR...NEXT loop or, when used with RESUME, continues execution from an error-trapping handler.

See Also «FOR...NEXT» «RESUME»

STEP Keyword

In a FOR...NEXT loop, specifies how much to increase the counter in each iteration. In graphics statements, specifies that pixel coordinates are relative to the current graphics cursor position.

See Also «CIRCLE» «FOR...NEXT» «GET, PUT» «LINE» «PAINT» «PRESET, PSET»

TO Keyword

Specifies ranges for:

- A CASE clause of a SELECT CASE statement.
- A loop counter in a FOR...NEXT loop.
- Records to lock or unlock in a LOCK...UNLOCK statement.
- Lower and upper bounds in a DIM or REDIM statement.

See Also «DIM, REDIM» «FOR...NEXT» «LOCK, UNLOCK» «SELECT CASE»

CLEAR Statement

Closes all files, releases file buffers, clears all common variables, sets numeric variables and arrays to zero, sets string variables to null, and initializes the stack. Optionally, CLEAR also changes the size of the stack.

CLEAR [,,stack&]

- **stack&** Sets the size (in bytes) of stack space for your program.

Example:

```
CLEAR ,,2000
```

See Also «ERASE»

DEF SEG Statement

Sets the current segment address.

DEF SEG [=address]

- **address** A segment address used by BLOAD, BSAVE, CALL ABSOLUTE, PEEK, or POKE; a value in the range 0 through 65,535. If address is omitted, DEF SEG resets the current segment address to the default data segment.

Example:

```
DEF SEG = 0
Status% = PEEK(&H417)              'Read keyboard status.
POKE &H417, (Status% XOR &H40)     'Change Caps Lock state, bit 6.
```

See Also «BSAVE, BLOAD» «CALL ABSOLUTE» «PEEK, POKE»

FRE Function

Returns the amount (in bytes) of available or unused memory.

FRE(numeric-expression)
FRE(stringexpression$)

- **numeric-expression** A value that specifies the type of memory:

Value	FRE returns
-1	The size of the largest array (nonstring) you can create
-2	The unused stack space
Any other number	The available string space

- **stringexpression$** Any string expression. FRE compacts the free string space into a single block, then returns the amount of available string space.

Example:

```
PRINT "String Space", FRE("")
PRINT "Unused Stack Space", FRE(-2)
PRINT "Array Space", FRE(-1)
```

HEX$, OCT$ Functions

HEX$ returns a hexadecimal string representation of a number.
OCT$ returns an octal string representation of a number.

HEX$(numeric-expression&)
OCT$(numeric-expression&)

- **numeric-expression&** Any numeric expression. The expression is rounded to an integer or long integer before it is evaluated.

Example:

```
INPUT x
a$ = HEX$ (x)
b$ = OCT$ (x)
PRINT x; "decimal is "; a$; " hexadecimal and "; b$; " in octal."
```

INSTR Function

Returns the position of the first occurrence of a string in another string.

INSTR([start%,]stringexpression1$,stringexpression2$)

- **start%** Sets the character position where the search begins. If start% is omitted, INSTR starts at position 1.
- **stringexpression1$** The string to search.
- **stringexpression2$** The string to look for.

Example:

```
a$ = "Microsoft QBasic"
PRINT "String position ="; INSTR(1, a$, "QBasic")
```

See Also «LEFT$, RIGHT$» «LEN» «MID$»

LCASE$, UCASE$ Functions

«Contents» «Index» «Back»

Convert strings to all lowercase or all uppercase letters.

LCASE$(stringexpression$)
UCASE$(stringexpression$)

- **stringexpression$** Any string expression.

Example:

```
Test$ = "THE string"
PRINT Test$
PRINT LCASE$(Test$); " in lowercase"
PRINT UCASE$(Test$); " IN UPPERCASE"
```

LEFT$, RIGHT$ Functions

«Contents» «Index» «Back»

Return a specified number of leftmost or rightmost characters in a string.

```
LEFT$(stringexpression$,n%)
RIGHT$(stringexpression$,n%)
```

- **stringexpression$** Any string expression.
- **n%** The number of characters to return, beginning with the leftmost or rightmost string character.

```
Example:
    a$ = "Microsoft QBasic"
    PRINT LEFT$(a$, 5)     'Output is:  Micro
    PRINT RIGHT$(a$, 5)    'Output is:  Basic
```

See Also «MID$»

LEN Function

Returns the number of characters in a string or the number of bytes required to store a variable.

```
LEN(stringexpression$)
LEN(variable)
```

- **stringexpression$** Any string expression.
- **variable** Any nonstring variable.

```
Example:
    a$ = "Microsoft QBasic"
    PRINT LEN(a$)
```

See Also «OPEN»

LTRIM$, RTRIM$ Functions

Remove leading and trailing spaces from a string.

```
LTRIM$(stringexpression$)
RTRIM$(stringexpression$)
```

- **stringexpression$** Any string expression.

```
Example:
    a$ = "    Basic    "
    PRINT "*" + a$ + "*"            'Output is:  *    Basic    *
    PRINT "*" + LTRIM$(a$) + "*"    'Output is:  *Basic    *
    PRINT "*" + RTRIM$(a$) + "*"    'Output is:  *    Basic*
```

MID$ Function and Statement

«Contents» «Index» «Back»

The MID$ function returns part of a string (a substring).
The MID$ statement replaces part of a string variable with another string.

MID$(stringexpression$,start%[,length%])
MID$(stringvariable$,start%[,length%])=stringexpression$

- **stringexpression$** The string from which the MID$ function returns a substring, or the replacement string used by the MID$ statement. It can be any string expression.
- **start%** The position of the first character in the substring being returned or replaced.
- **length%** The number of characters in the substring. If the length is omitted, MID$ returns or replaces all characters to the right of the start position.
- **stringvariable$** The string variable being modified by the MID$ statement.

```
Example:
    a$ = "Where is Paris?"
    PRINT MID$(a$, 10, 5)       'Output is:  Paris
    Text$ = "Paris, France"
    PRINT Text$                 'Output is:  Paris, France
    MID$(Text$, 8) = "Texas "
    PRINT Text$                 'Output is:  Paris, Texas
```

See Also «LEFT$, RIGHT$» «LEN»

MKn$, CVn Functions

«Contents» «Index» «Back»

MKI$, MKL$, MKS$, and MKD$ convert numbers to numeric strings that can be

stored in FIELD statement string variables. CVI, CVL, CVS, and CVD convert those strings back to numbers.

MKI$(integer-expression%)
MKL$(long-integer-expression&)
MKS$(single-precision-expression!)
MKD$(double-precision-expression#)
CVI(2-byte-numeric-string)
CVL(4-byte-numeric-string)
CVS(4-byte-numeric-string)
CVD(8-byte-numeric-string)

Function	Returns	Function	Returns
MKI$	A 2-byte string	CVI	An integer
MKL$	A 4-byte string	CVL	A long integer
MKS$	A 4-byte string	CVS	A single-precision number
MKD$	An 8-byte string	CVD	A double-precision number

See Also «FIELD» «MKSMBF$, MKDMBF$, CVSMBF, CVDMBF»

MKSMBF$, MKDMBF$, CVSMBF, CVDMBF Functions

«Contents» «Index» «Back»

MKSMBF$ and MKDMBF$ convert IEEE-format numbers to Microsoft-Binary-format numeric strings that can be stored in FIELD statement string variables. CVSMBF and CVDMBF convert those strings back to IEEE-format numbers.

MKSMBF$(single-precision-expression!)
MKDMBF$(double-precision-expression#)
CVSMBF (4-byte-numeric-string)
CVDMBF (8-byte-numeric-string)

Function	Returns
MKSMBF$	A 4-byte string containing a Microsoft-Binary-format number
MKDMBF$	An 8-byte string containing a Microsoft-Binary-format number
CVSMBF	A single-precision number in IEEE format
CVDMBF	A double-precision number in IEEE format

- These functions are useful for maintaining data files created with older versions of Basic.

Example:

```
TYPE Buffer
    SngNum AS STRING * 4
    DblNum AS STRING * 8
```

```
    END TYPE
    DIM RecBuffer AS Buffer
    OPEN "TESTDAT.DAT" FOR RANDOM AS #1 LEN = 12
    SNum = 98.9
    DNum = 645.3235622#
    RecBuffer.SngNum = MKSMBF$(SNum)
    RecBuffer.DblNum = MKDMBF$(DNum)
    PUT #1, 1, RecBuffer
    GET #1, 1, RecBuffer
    CLOSE #1
    PRINT CVSMBF(RecBuffer.SngNum), CVDMBF(RecBuffer.DblNum)
```

See Also «FIELD» «MKn$, CVn»

PEEK Function, POKE Statement

PEEK returns a byte value stored at a specified memory location.
POKE writes a byte value to a specified memory location.

PEEK(address)
POKE address,byte%

- **address** A byte position relative to the current segment address set by DEF SEG; a value in the range 0 through 65,535.
- **byte%** A byte value to write to the specified memory location; a value in the range 0 through 255.

Example:

```
    DEF SEG = 0
    Status% = PEEK(&H417)              'Read keyboard status.
    POKE &H417, (Status% XOR &H40)     'Change Caps Lock state, bit 6.
```

See Also «DEF SEG»

SPACE$ Function

Returns a string of spaces.

SPACE$(n%)

- **n%** The number of spaces you want in the string.

```
Example:
    FOR i% = 1 TO 5
        x$ = SPACE$ (i%)
        PRINT x$; i%
    NEXT i%
```

See Also «SPC» «STRING$»

STR$, VAL Function

STR$ returns a string representation of a number.
VAL converts a string representation of a number to a number.

STR$(numeric-expression)
VAL(stringexpression$)

- **numeric-expression** Any numeric expression.
- **stringexpression$** A string representation of a number.

```
Example:
    PRINT "Decimal 65 is represented in hexadecimal as ";
    PRINT "&H" + LTRIM$(STR$(41))
    PRINT VAL(RIGHT$("Microsoft 1990", 4))
```

STRING$ Function

Returns a string of a specified length made up of a repeating character.

STRING$(length%,{ascii-code% | stringexpression$})

- **length%** The length of the string.
- **ascii-code%** The ASCII code of the repeating character.
- **stringexpression$** Any string expression. STRING$ fills the string with the first character in stringexpression$.

Example:

```
    PRINT STRING$(5, "-");
    PRINT "Hello";
    PRINT STRING$(5, "-")
```

See Also «ASCII Character Codes»

VARPTR, VARSEG Functions

VARPTR returns the offset address of a variable.
VARSEG returns the segment address of a variable.

VARPTR(variablename)
VARSEG(variablename)

- **variablename** Any Basic variable.

See Also «CALL ABSOLUTE» «DEF SEG» «PEEK, POKE» «VARPTR$»

VARPTR$ Function

Returns a string representation of a variable's address for use in DRAW and PLAY statements.

VARPTR$(commandstring$)

- **commandstring$** A string variable containing DRAW or PLAY commands.

Example:

```
    Scale$ = "CDEFGAB"
    PLAY "L16"
    FOR i% = 0 TO 6
        PLAY "O" + STR$(i%)
        PLAY "X" + VARPTR$(Scale$)
    NEXT i%
```

See Also «DRAW» «PLAY (Music)» «VARPTR, VARSEG»

ERR, ERL Functions

ERR returns the run-time error code for the most recent error.
ERL returns the line number where the error occurred, or the closest line number before the line where the error occurred.

ERR
ERL

- ERL does not return line labels. If there are no line numbers in the program, ERL returns 0.

«Example»

See Also «ERDEV, ERDEV$» «ERROR» «ON ERROR» «RESUME» «Run-Time Error Codes»

ERROR Statement

Simulates an occurrence of a Basic error or a user-defined error.

ERROR expression%

- **expression%** The error code of a Basic or user-defined error; a value in the range 1 through 255. To define your own error, use a value that isn't listed in the Basic «Run-Time Error Codes» table.

«Example»

See Also «ERDEV, ERDEV$» «ERR, ERL» «ON ERROR» «RESUME»

ON ERROR Statement

Enables error handling and, when a run-time error occurs, directs your program to either branch to an error-handling routine or resume execution.

ON ERROR {GOTO line | RESUME NEXT}

- **GOTO line** Branches to the first line of the error-handling routine, specified by a label or line number. To disable error handling, specify GOTO 0.
- **RESUME NEXT** Resumes execution with the statement following the statement that caused the run-time error. Use the ERR function to obtain the error code for the error.

- If ON ERROR isn't used, any run-time error ends your program.

«Example»

See Also «ERDEV, ERDEV$» «ERR, ERL» «ERROR» «GOTO» «RESUME»

RESUME Statement

«Contents» «Index» «Back»

Resumes program execution after an error-handling routine.

RESUME [{line | NEXT}]

- **line** The label or number of the line where execution resumes. If line is 0 or omitted, execution resumes with the statement that caused the error.
- **NEXT** Resumes execution at the statement following the statement that caused the error.

«Example»

See Also «ERROR» «ON ERROR»

Error-Handling Example

«Contents» «Index» «Back»

```
'Illustrates ERDEV, ERDEV$, ERL, ERR, ERROR, ON ERROR, and RESUME.
   ON ERROR GOTO Handler
10 CHDIR "a:\\"                 'Causes ERR 71 "Disk not ready"
                               'if no disk in Drive A.
20 y% = 0
30 x% = 5 / y%                  'ERR 11 "Division by zero."
40 PRINT "x% ="; x%
50 ERROR 57                     'ERR 57 "Device I/O error."


Handler:
  PRINT
  PRINT "Error "; ERR; " on line "; ERL
  SELECT CASE ERR
    CASE 71
      PRINT "Using device "; ERDEV$; " device error code = "; ERDEV
      RESUME NEXT
    CASE 11
      INPUT "What value do you want to divide by"; y%
      RESUME                    'Retry line 30 with new value of y%.
    CASE ELSE
      PRINT "Unexpected error, ending program."
      END
  END SELECT
```

QBasic Online Help Index

To get help on a QBasic keyword in the list below:

1. Press the key of the first letter of the keyword.
2. Use the direction keys to move the cursor to the keyword.
3. Press F1 to display the help text in the Help window.

+===+
+ A +
+===+

ABS Function
ABSOLUTE Keyword
ACCESS Keyword
AND Operator
ANY Keyword
APPEND Keyword
AS Keyword
ASC Function
ATN Function

+===+
+ B +
+===+

BASE Keyword
Basic Character Set
BEEP Statement
BINARY Keyword
BLOAD Statement
Boolean Operators
BSAVE Statement

+===+
+ C +
+===+

CALL Statement
CALL ABSOLUTE Statement
CASE Keyword
COLOR Statement
COM Statement
COMMON Statement

CDBL Function
CHAIN Statement
CHDIR Statement
CHR$ Function
CINT Function
CIRCLE Statement
CLEAR Statement
CLNG Function
CLOSE Statement
CLS Statement
CONST Statement
COS Function
CSNG Function
CSRLIN Function
CVD Function
CVDMBF Function
CVI Function
CVL Function
CVS Function
CVSMBF Function

D

DATA Statement
Data Type Keywords
DATE$ Function
DATE$ Statement
DECLARE Statement
DEF FN Statement
DEF SEG Statement
DEFDBL Statement
DEFINT Statement
DEFLNG Statement
DEFSNG Statement
DEFSTR Statement
DIM Statement
DO...LOOP Statement
DOUBLE Keyword
DRAW Statement
$DYNAMIC Metacommand

E

ELSE Keyword
ELSEIF Keyword
END Statement
ENVIRON Statement
ENVIRON$ Function
EOF Function
EQV Operator
ERASE Statement
ERDEV Function
ERDEV$ Function
ERL Function
ERR Function
ERROR Statement
EXIT Statement
EXP Function

F

FIELD Statement
FILEATTR Function
FILES Statement
FIX Function
FOR...NEXT Statement
FRE Function
FREEFILE Function
FUNCTION Statement

G

GET (File I/O) Statement
GET (Graphics) Statement
GOSUB Statement
GOTO Statement

H

HEX$ Function

I

IF...THEN...ELSE Statement
IMP Operator
INKEY$ Function
INP Function
INPUT Statement
INPUT$ Function
INSTR Function
INT Function
INTEGER Keyword
IOCTL Statement
IOCTL$ Function
IS Keyword

K

KEY (Assignment) Statement
KEY (Event Trapping) Statement
KILL Statement

L

LBOUND Function
LCASE$ Function
LEFT$ Function
LEN Function
LET Statement
LINE (Graphics) Statement
LINE INPUT Statement
LIST Keyword
LOC Function
LOCATE Statement
LOCK...UNLOCK Statements
LOF Function
LOG Function
LONG Keyword
LOOP Keyword
LPOS Function
LPRINT Statement
LPRINT USING Statement
LSET Statement
LTRIM$ Function

M

MID$ Function
MID$ Statement
MKD$ Function
MKDIR Statement
MKDMBF$ Function
MKI$ Function
MKL$ Function
MKS$ Function
MKSMBF$ Function
MOD Operator

N

NAME Statement
NEXT Keyword
NOT Operator

O

OCT$ Function
OFF Keyword
ON COM Statement
ON ERROR Statement
ON Keyword
ON KEY Statement
ON PEN Statement
ON PLAY Statement
ON STRIG Statement
ON TIMER Statement
ON...GOSUB Statement
ON...GOTO Statement
OPEN Statement
OPEN COM Statement
OPTION BASE Statement
OR Operator
OUT Statement
OUTPUT Keyword

P

PAINT Statement
PALETTE Statements
PCOPY Statement
PEEK Function
PEN Function
PEN Statement
PLAY Function
PLAY (Music) Statement
PLAY (Event Trapping) Statements
PMAP Function
POINT Function
POKE Statement
POS Function
PRESET Statement
PRINT Statement
PRINT USING Statement
PSET Statement
PUT (File I/O) Statement
PUT (Graphics) Statement

R

RANDOM Keyword
RANDOMIZE Statement
READ Statement
RETURN Statement
RIGHT$ Function
RMDIR Statement

REDIM Statement
REM Statement
RESET Statement
RESTORE Statement
RESUME Statement
RND Function
RSET Statement
RTRIM$ Function
RUN Statement

S

SCREEN Function
SCREEN Statement
SEEK Function
SEEK Statement
SELECT CASE Statement
SGN Function
SHARED Statement
SHELL Statement
SIN Function
SINGLE Keyword
SLEEP Statement
SOUND Statement
SPACE$ Function
SPC Function
SQR Function
STATIC Statement
$STATIC Metacommand
STEP Keyword
STICK Function
STOP Statement
STR$ Function
STRIG Function
STRIG Statements
STRING Keyword
STRING$ Function
SUB Statement
SWAP Statement
SYSTEM Statement

T

TAB Function
TAN Function
THEN Keyword
TIME$ Function
TIME$ Statement
TIMER Function
TIMER Statements
TO Keyword
TROFF Statement
TRON Statement
TYPE Statement

U

UBOUND Function
UCASE$ Function
UNLOCK Statement
UNTIL Keyword
USING Keyword

V

VAL Function
VARPTR Function
VARPTR$ Function
VARSEG Function
VIEW Statement
VIEW PRINT Statement

W

WAIT Statement
WEND Keyword
WHILE...WEND Statement
WIDTH Statements
WINDOW Statement
WRITE Statement

X

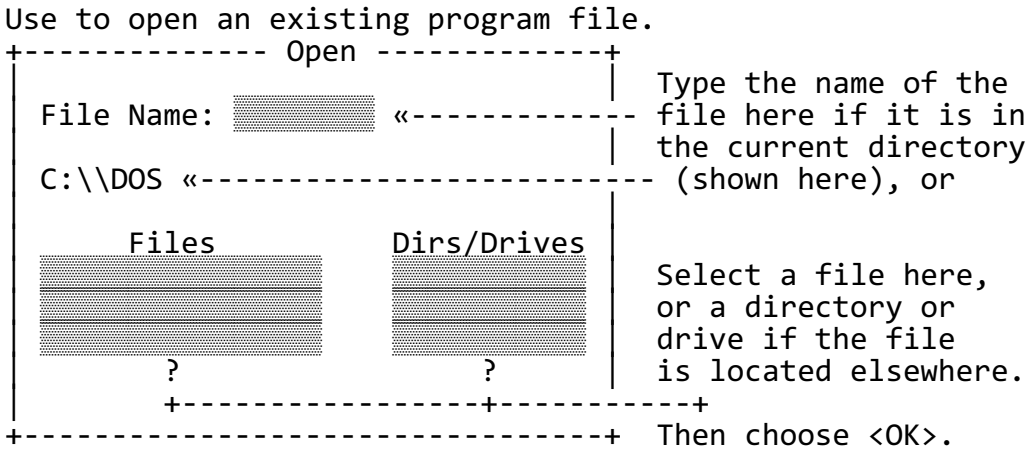
XOR Operator

Welcome to QBasic

- Press Esc to enter the QBasic programming environment.

- Press Enter to display the QBasic Survival Guide. The Survival Guide shows you how to start using the QBasic environment and the QBasic Help system.

Open Dialog

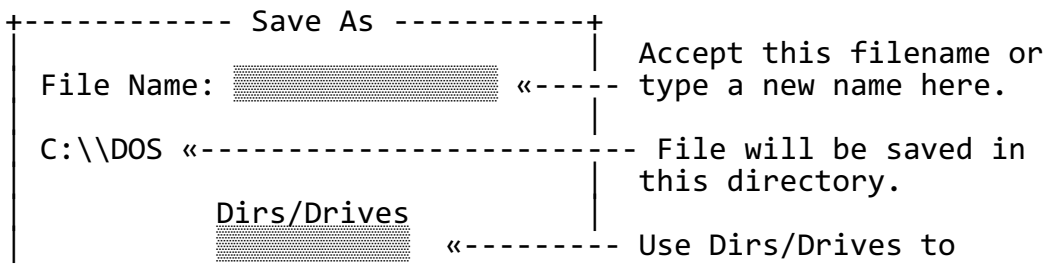


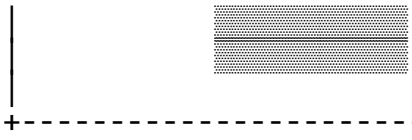
You can also select a new drive and directory by typing them in the File Name text box.

To browse the contents of other drives and directories, type a wildcard filename (such as *.BAS) in the File Name text box, and select a new drive or directory.

Note: To open a file created with GW-BASIC or BASICA, the file must be saved from GW-BASIC or BASICA with the ,A option.

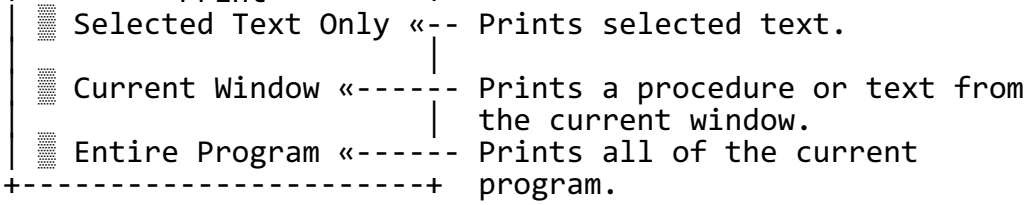
Save As Dialog





Print Dialog

Use to print all or part of a program or Help topic.
Make sure the printer attached to the LPT1 device
on your computer is on and ready to print. Then choose
one of the following options and press Enter:



The Entire Program option is not available if you select
Print while the cursor is in the Help window.

Tip: Select text in your program or the Help window
before you choose the Print command. Place the
cursor at the beginning of the text you want to
print, press Shift+direction keys until all the text
is highlighted, then choose the Print command.

New SUB or FUNCTION Dialog

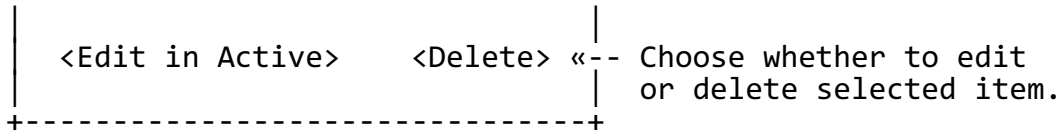
Use to create a new SUB or FUNCTION procedure.
Enter the name of the new procedure (up to 40 characters)
in the text box, then choose <OK>.

Descriptive procedure names help you remember the
procedure's use and minimize the chances of duplicating
the name of another procedure or a variable.

SUBs Command Dialog

Use to edit or delete SUB and FUNCTION procedures.

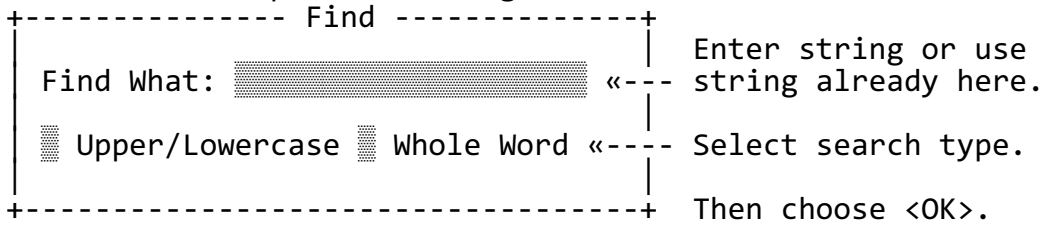




If you choose <Edit in Active>, QBasic displays the procedure in the active window.

Find Command Dialog

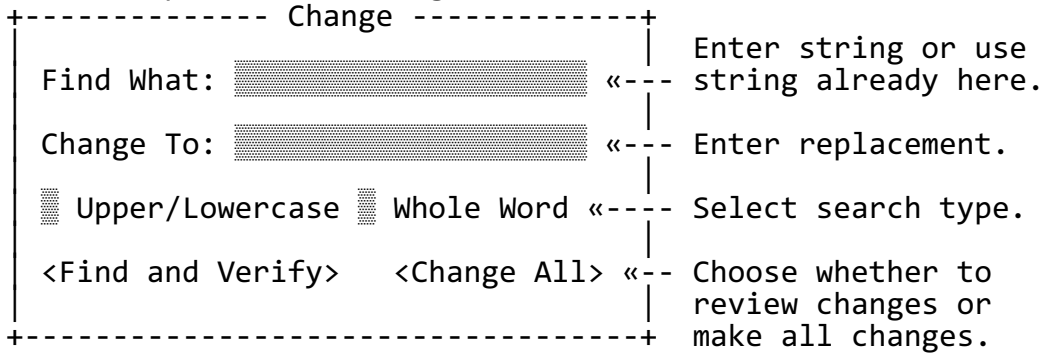
Use to find a specific string of text.



Select Upper/Lowercase for a case-sensitive match, or Whole Word to exclude matches within words or strings. Whole-word searches recognize a match only when the target text is surrounded by blanks, punctuation, or other special characters. The search string can be 127 characters long.

Change Dialog

Use to replace one string of text with another.



Select Upper/Lowercase for a case-sensitive match, or Whole Word to exclude matches within words or strings. Whole-word searches recognize a match only when the target text is surrounded by blanks, punctuation, or other special characters. Replacement text can be 127 characters long.

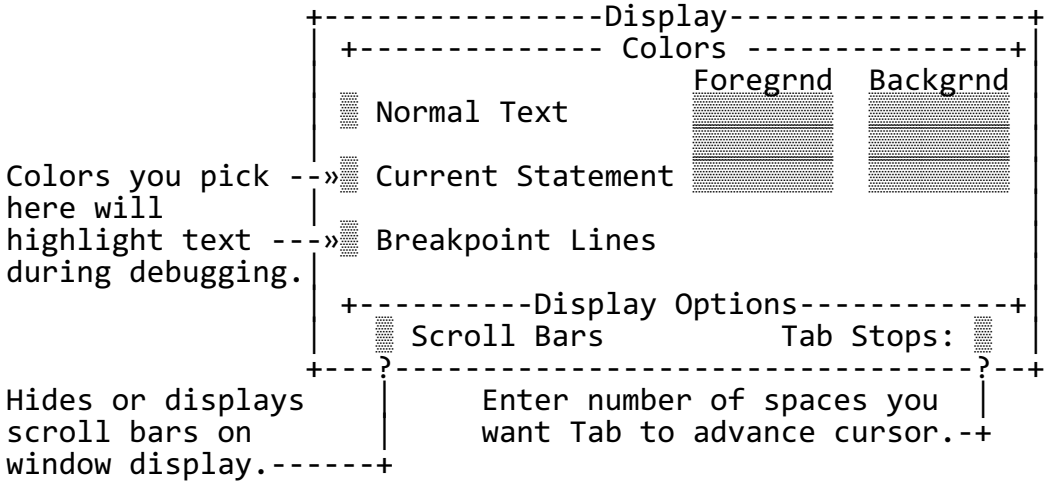
Change Dialog

A match has been found and is highlighted in the active window. You can:

- Choose <Change> to make the replacement and continue searching.
- Choose <Skip> to leave the text unchanged and continue searching.
- Choose <Cancel> to leave the text unchanged and stop searching.

Display Dialog

Use to customize the QBasic screen.



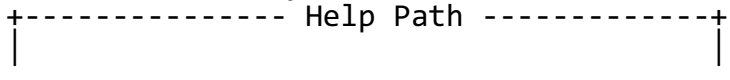
To set options:

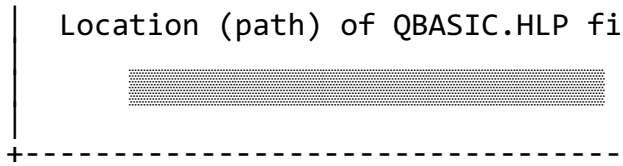
1. Press Tab to move the cursor to the Colors or Display Options box.
2. Use the direction keys to select a color or display option.
3. Choose <OK> or press Enter to save the options you selected.

Options settings are saved and remain in effect the next time you start QBasic.

Help Path Dialog

QBasic searches for the help file QBASIC.HLP in the directories you indicate in this dialog box. You don't need to set the Help path if you have done so with the PATH environment variable in DOS or if QBASIC.HLP is in the current directory.





Tip: If QBasic cannot find QBASIC.HLP, check to make
sure that you entered the correct directory name.

QBasic Menu Bar

Using the QBasic menus and commands:

- To activate menus and commands, press the highlighted letter or click a menu or command with the mouse.
- To move between menus and commands, use the direction keys or the mouse.

New Command

Use to create a new program. A dialog box will appear if you already have an existing program open. Then:

- Choose <Yes> to save the existing program.
- Choose <No> to close the program without saving it.
- Choose <Cancel> to cancel the New command and return to the existing program.

Tip: Use the Open command to load an existing program.

Open Command

Use to load an existing program so you can run it or make changes to it.

Use the New command to create a new program.

Note: To open a file created with GW-BASIC or BASICA, the file must be saved from GW-BASIC or BASICA with the ,A option.

Save Command

Use to save the current version of your program.

QBasic asks you for a name if you haven't saved the program previously. If an existing program has the same name as the program being saved, the existing program is overwritten.

Save As Command

Use to save your program as a file. To preserve the previous version of your program, rename it in the File Name text box.

Print Command

Use to print all or part of a program or Help topic.

You can print:
- Selected text.
- The contents of the active window.
- The entire current program (including procedures).

QBasic prints to the printer connected to the LPT1 device on your computer.

Tip: Select text in your program or the Help window before you choose the Print command. Place the cursor at the beginning of the text you want to print, press Shift+direction keys until all the text is highlighted, then choose the Print command.

Exit Command

Use to leave the QBasic environment.

If your program file has changed since you last saved it, QBasic displays a dialog box.
- Choose <Yes> to save your program, including changes.
- Choose <No> to discard your changes.
- Choose <Cancel> to cancel the Exit command and return to your program.

Cut Command

Use Cut (or press Shift+Del) to remove selected text and put it on the Clipboard, a temporary holding area.

You can then use Paste (Shift+Ins) to insert the text at a new position in the active window. You can also open a new or existing program file and insert the text.

The text is still on the Clipboard and can be pasted again. Each time you cut or copy a piece of text, the new text replaces any text on the Clipboard.

Tip: To select text, place the cursor at the beginning of the text and press the Shift+direction keys. To remove selected text without copying it to the Clipboard, press Del.

Copy Command

Use Copy (or press Ctrl+Ins) to copy selected text to the Clipboard. The original text is unchanged.

You can then use Paste (Shift+Ins) to insert the text into the active window. You can also open a new or existing program file and insert the text.

Tip: To select text, place the cursor at the beginning of the text and press the Shift+direction keys.

The text is still on the Clipboard and can be pasted again. Each time you cut or copy a piece of text, the new text replaces any text on the Clipboard.

Paste Command

Use Paste (or press Shift+Ins) to insert a block of text from the Clipboard at any point in the active window.

To insert text, place the cursor where you want the text to be inserted and choose Paste.

To replace one piece of text with another, select the text to replace and choose Paste.

Tip: To put text on the Clipboard, use Cut or Copy. After you insert text using Paste, the Clipboard still contains the text, and it can be pasted again.

Clear Command

Use Clear (or press Del) to delete selected text without copying it to the Clipboard. The Clipboard's contents are unchanged.

New SUB Command

Use to create a SUB procedure.

Enter a name for the procedure in the New SUB dialog box. QBasic inserts SUB and END SUB statements for the procedure and displays it in the View window.

If you want to view or edit other parts of the program, use the SUBs command from the View menu (or press F2).

Tip: You can also create a SUB procedure by using the QBasic keyword SUB.

New FUNCTION Command

Use to create a FUNCTION procedure.

Enter a name for the procedure in the New FUNCTION dialog box. QBasic inserts FUNCTION and END FUNCTION statements for the procedure and displays it in the View window.

If you want to view or edit other parts of the program, use the SUBs command from the View menu (or press F2).

Tip: You can also create a FUNCTION procedure by using the QBasic keyword FUNCTION.

SUBs Command

Use to view or edit a SUB or FUNCTION procedure, or to delete a procedure.

Tip: To view or edit two different SUB or FUNCTION procedures at the same time, choose Split from the

View menu, then use the SUBs command to display the procedures in each of the windows.

Split Command

Use to divide the View window horizontally and work on two parts of a program at the same time.

Use F6 and Shift+F6 to move between the windows:

- F6 moves the cursor down through the windows.
- Shift+F6 moves the cursor up.

To close a second View window, place your cursor in the window that you want to keep active, and choose Split again.

To size the active window with a mouse, drag the window boundary up or down.

To size the active window from the keyboard, use:

- Alt+Plus(+) to expand it one line.
- Alt+Minus(-) to shrink it one line.
- Ctrl+F10 to expand it to fill the screen, or return it to its former size.

Output Screen Command

Use Output Screen (or press F4) to view the program output screen. The output screen displays the results of a program run in the QBasic environment.

Press any key to return to QBasic after viewing the output screen.

Find Command

Use to search for a text string. You can request a case-sensitive match or a whole-word match.

To change a string, use the Change command.

Change Command

Use to replace one text string with another.

You can:
- Use a case-sensitive or whole-word search.
- Check each replacement before the change is made.
- Make all replacements without checking them.

Repeat Last Find Command

Use Repeat Last Find (or press F3) to repeat the search performed by the most recent Find or Change command.

If no Find or Change command has been executed since you started QBasic, Repeat Last Find searches for the next occurrence of:
- The word the cursor is on.
- The word to the cursor's left, if the cursor isn't on a word.

Start Command

Use Start (or press Shift+F5) to start running a program beginning with the first executable statement in the main module.

If you stop a program while it is running, Start clears any data you entered and starts the program from its beginning.

Restart Command

Use to prepare a program to start over from the beginning. Restart clears any data entered while the program was running and highlights the first executable statement.

Continue Command

Use Continue (or press F5) to resume running a program after it has stopped. Continue does not clear data you entered while the program was running and begins the program from the last statement executed, rather than from the beginning.

Continue is often used to resume program execution after a breakpoint.

Step Command

Use Step (or press F8) to run a single statement in your program. Step allows you to move through the execution of a program, including procedures, one step at a time.

Procedure Step Command

Use Procedure Step (or press F10) to run a single statement in your program and to run a procedure call and the procedure it calls as a single statement.

Procedure Step, like the Step command, allows you to move through the execution of a program one step at a time, but it saves you from stepping through procedures that you know work correctly.

Trace On Command

Use to highlight each statement in your program as it executes. This lets you observe the general flow of your program. Press Ctrl+Break to suspend program execution.

Toggle Breakpoint Command

Use Toggle Breakpoint (or press F9) to turn breakpoints on and off. Breakpoints are markers placed on statements in your program. If you run your program and it encounters a breakpoint, the program stops on that statement. QBasic highlights the breakpoint line.

To turn a breakpoint on and off:

1. Move the cursor to the line you want the breakpoint on, or to a highlighted breakpoint line.
2. Choose Toggle Breakpoint (or press F9).

Use a breakpoint to pause your program at a point where you suspect problems, then:

- Print the values of variables in the Immediate window.
- Single step your program from the breakpoint forward.

Clear All Breakpoints Command

Use to remove all breakpoints from your program.

Use Toggle Breakpoint (or press F9) to turn individual breakpoints off.

Set Next Statement Command

Use to change the program execution sequence so the next statement executed is the one the cursor is on.

Display Command

Use to control:

- Screen colors.
- Scroll bars in windows.
- The number of spaces the Tab key advances the cursor.

Display settings are saved and remain in effect the next time you start QBasic.

Help Path Command

Use to change the directories QBasic searches to find the QBasic Help file, QBASIC.HLP.

Syntax Checking Command

Use to turn Syntax Checking on and off. When Syntax Checking is on, QBasic does the following as you enter each line of code:

1. Checks the line for syntax errors.
2. Formats the line.
3. Translates the line to executable form if the syntax is correct.

Index Command

Use to display the Help index. The Help index is an alphabetical list of Help topics, including Basic

language keywords. Each term in the index is linked to additional information.

To get help:

1. Choose the Index command from the Help menu.
2. Press the key corresponding to the item's first letter.
3. Move the cursor to the item you want help on, then press F1.

Contents Command

Use to display the Help table of contents. The Help table of contents provides a topical guide to the information in QBasic Help.

To get help on any item in the table of contents:

1. Choose the Contents command from the Help menu.
2. Press the letter key corresponding to the first letter of the item. Repeat until the cursor is on the topic you want help on.
3. Press F1.

Topic Command

Use to display information on a topic determined by the current location of the cursor. The topic the cursor is on appears as part of the command.

You can also display topic Help by pressing F1 or clicking the right mouse button.

Using Help Command

Use to display the Using Help topic. Using Help describes how to use the mouse or keyboard to get help on an item displayed on the QBasic screen.

About Command

Use to display the version number and copyright information for QBasic.

Translation momentarily stopped

QBasic has stopped doing an item-by-item translation of the program line at the highlighted item. The error is ordinarily to the left of the item. You could:

- Substitute an item suggested in the dialog box.
- Correct any misspelled keywords.
- Move the cursor to a keyword in the statement and press F1 to see the correct syntax for the keyword.

NEXT without FOR

Each NEXT statement must have a matching FOR statement.

ERR code: 1

Syntax error

The most common cause of this error is an incorrectly typed Basic keyword or argument. Check to make sure the program line is correct.

ERR code: 2

RETURN without GOSUB

The program just executed a RETURN statement without having executed a matching GOSUB statement.

ERR code: 3

Out of DATA

A READ statement has been executed but there are no DATA statements with unread data remaining in the program.

ERR code: 4

Illegal function call

You are attempting to give an improper or out-of-range argument to a Basic statement. Examples of such errors are:

- A negative number is raised to a power that is not an integer.
- A negative record number is given when using GET or PUT.
- An I/O function or statement (LOC or LOF, for example) is performed on a device that does not support it.
- Strings are concatenated to create a string greater than 32,767 characters in length.
- A BLOAD or BSAVE operation is directed to a non-disk device.

ERR code: 5

Overflow

The result of a calculation or data-type conversion is too large for the given data type.

ERR code: 6

Out of memory

More memory was required than is available. To correct this:

- Exit QBasic and reduce the number of DOS buffers, memory resident programs, or loaded device drivers. Then reboot your computer.
- Make your program smaller.
- Reduce the number or size of arrays.
- Reduce the stack size by using the CLEAR statement.
- Reduce the number of variables.

ERR code: 7

Label not defined

A line label is referred to (in a GOTO statement, for example), but does not occur in the program.

ERR code: 8

Subscript out of range

An array element was referred to with a subscript that was outside the dimensions of the array, or an element of an undimensioned dynamic array was accessed. Check that the array is dimensioned and check the bounds of the array.

You may also get this error if the array size exceeds 64K. Reduce the size of the array.

ERR code: 9

Duplicate definition

This error occurs if you try to change the dimensions of a static array while your program is running. To correct this, declare the array as dynamic using the REDIM statement.

This error also occurs if you try to change the dimensions of a dynamic array using the DIM statement without first deallocating the array. To correct this, add an ERASE statement before dimensioning the array.

ERR code: 10

Duplicate definition

You are attempting to create a new element in your program and have given it a name that is already being used.

For example:

- A CONST statement uses the same name as an existing variable.
- A new variable or procedure has the same name as an existing procedure.

Starting a variable name with the letters "FN" is another way to generate this error, because FN is used exclusively with DEF FN.

Division by zero

This error can be caused by dividing by zero in an expression.

ERR code: 11

Illegal in direct mode

The highlighted statement is valid only within a program and cannot be used in the Immediate window. In general, the following statements and metacommands cannot be used in the Immediate window:

- COMMON, CONST, DATA, DECLARE, DIM, OPTION, SHARED, STATIC, and TYPE
- $DYNAMIC, $STATIC metacommands
- DEF FN...END DEF, ELSE IF, END IF, END TYPE FUNCTION...END FUNCTION, REDIM, and SUB...END SUB

ERR code: 12

Type mismatch

The variable is not of the required type. For example, you are trying to use the SWAP statement with a string variable and a numeric variable, or with two numeric variables of different types.

ERR code: 13

Out of string space

The string space needed for your string variables exceeds the amount available. To get more string space:

- Reduce the number of variables.
- Reduce the number or size of arrays of strings.
- Reduce the stack size. Use the CLEAR statement to resize the stack.

ERR code: 14

String formula too complex

Either a string formula is too long, or you have specified more than 15 string variables in an INPUT statement. Break the formula or INPUT statement into smaller parts.

ERR code: 16

QBasic cannot continue

Cannot continue is a prompt that may occur while you are debugging. While your program was suspended (at a breakpoint, for example) you made a change to the program that has implications throughout the program. You may have changed the dimensions of an array, changed procedure arguments, or edited some other declarative statement. Use the dialog box and either:

- Choose <OK> to keep the change and restart the program.
- Choose <Cancel> to undo the change in your program text. (You can do this if restarting interferes with your debugging strategy.)

User-Generated Error

You have used an ERROR statement to generate this error.

ERR codes: 17, 18, 35, 39

QBasic function not defined

The function defined by DEF FN does not exist, which can indicate one of the following conditions:

- The DEF FN function is undefined.
- The DEF FN function is attempting to call itself recursively.

No RESUME

The end of the program was encountered while the program was executing an error-handling routine. Add a RESUME statement in the error-handling routine.

ERR code: 19

RESUME without error

A RESUME statement was executed, but there is no active error-handling routine.

ERR code: 20

Device timeout

The program did not receive information from a device, such as the printer, within the timeout period.

ERR code: 24

Device timeout

QBasic did not receive information from a device, such as the printer, within the timeout period:

- If you are attempting to print, make sure the printer is attached to the device LPT1.

Device fault

A device has returned a hardware error, which can indicate one of the following conditions:

- If you are attempting to print a file, the printer is not attached to the device LPT1.
- If you are transmitting data to a communications file, the signals being tested with the OPEN COM statement were not received within the timeout period.

ERR code: 25

QBasic device fault

QBasic has detected a fault at the printer. Make sure that:

- The printer online indicator light is still on.
- The printer power is still on.
- The printer cable is securely connected to the printer and the computer.

FOR without NEXT

Each FOR statement must have a matching NEXT statement.

ERR code: 26

Out of paper

The printer is out of paper, or it is not turned on.

ERR code: 27

WHILE without WEND

Each WHILE statement must have a matching WEND statement.

ERR code: 29

WEND without WHILE

Each WEND statement must have a matching WHILE statement.

Also, verify that other control structures within the WHILE...WEND structure are correctly matched. For example, an IF without a matching ENDIF inside the WHILE...WEND structure will generate this error.

ERR Code: 30

Duplicate label

Two program lines were assigned the same line number or label. Each line number or label must be unique.

Change the highlighted line number or label, or use the Find command from the Search menu to locate the duplicate number or label, then change that number or label to be unique.

ERR code: 33

QBasic subprogram not defined

You are attempting to call a SUB procedure that QBasic cannot find.

Press F2 to view the SUB and FUNCTION procedures that are available.

Argument-count mismatch

An incorrect number of arguments was used in a SUB or FUNCTION procedure call.

Compare the DECLARE statement for the SUB or FUNCTION procedure with the CALL statement to make sure the argument list has the same number of items in both cases.

ERR Code: 37

Array not defined

Your program is attempting to use an array that is not currently defined.

Verify that the array name is correct, and define the array with a DIM or REDIM statement.

ERR code: 38

Variable required

QBasic may have encountered an INPUT, LET, READ, or SHARED statement without a variable argument. QBasic may also have encountered a GET or PUT statement which didn't specify a variable when an operation was performed on a file opened in binary mode.

ERR code: 40

FIELD overflow

A FIELD statement attempted to allocate more bytes than were specified for the record length of a random file. Check the OPEN statement for the file to verify the record length.

ERR code: 50

Internal error

An internal error occurred in QBasic that should be reported to Microsoft.

ERR code: 51

Bad file name or number

This error may occur because a statement or command refers to a file with a file number or name that is not specified in the OPEN statement or is out of the range of file numbers (valid range is 1-255) specified earlier in the program.

ERR code: 52

File not found

An OPEN or RUN or other Basic statement refers to a file that does not exist.

ERR code 53

QBasic File not found

In the Open command dialog box, you specified a file that does not exist. Check to see that the filename is spelled correctly, and that you chose the correct drive and directory.

ERR code 53

Bad file mode

The error may have occurred because:

- A PUT or GET statement specified a sequential file.
- A FIELD statement specified a file not opened for random access.
- A PRINT # statement specified a sequential file opened for input.
- An INPUT # statement specified a file opened for output or appending.
- You attempted to open a QuickBasic binary format file.

ERR code: 54

File already open

This error is caused by one of the following conditions:

- A sequential-output-mode OPEN statement has been executed for a file that is already open.
- A KILL statement refers to an open file.

ERR code: 55

FIELD statement active

A GET or PUT statement specified a record variable in a file for which FIELD statements had been executed.

GET or PUT with a record-variable argument can be used only on a file if no FIELD statements have been executed for the file.

ERR code: 56

Device I/O error

An input or output error occurred while your program was using a device, such as the printer or disk drive.

ERR code: 57

QBasic device I/O error

You used the File menu's Print command and the printer malfunctioned. Check the printer.

File already exists

The filename specified in a NAME statement is identical to the name of a file that already exists. Specify another filename in the NAME statement, change directories, or rename or delete the existing file.

ERR code: 58

Bad record length

The length of a record variable in a GET or PUT statement did not match the record length specified in the corresponding OPEN statement.

ERR code: 59

Disk full

There wasn't enough room on the disk to complete a PRINT #, PRINT USING #, WRITE #, CLOSE, or PUT operation.

ERR code: 61

QBasic disk full condition

There isn't room on the specified disk to save the file. Save the file to another disk.

Input past end of file

An INPUT # statement read from an empty file or from a file in which all data has already been read.

To avoid this error, use the EOF function to detect the end of file.

ERR code: 62

Bad record number

The record number in a PUT or GET statement was less than or equal to zero.

ERR code: 63

Bad file name

A filename does not follow the appropriate naming convention for BLOAD, BSAVE, KILL, or OPEN (for example, the filename has too many characters).

ERR code: 64

Bad file name

The filename you specified either contains too many characters or characters that are not allowed in filenames.

Too many files

You have exceeded the number of file operations DOS allows in a root directory. If your program is opening, closing, and/or saving files in the root directory, change your program so it uses a subdirectory.

ERR code: 67

QBasic too many files limit

Your program has opened more disk files than QBasic can handle. You must close one or more of the open files before you can use QBasic commands.

The easiest way to do this is with the Basic CLOSE statement:

1. Move the cursor to the Immediate window.
2. Enter CLOSE on the first available text line.
3. Press Enter to execute the statement.

You can then execute QBasic commands. If your program was running, you will have to restart it.

Device unavailable

The device you are attempting to access is not online or does not exist.

ERR code: 68

QBasic device unavailable

You are attempting to open a file on a device that does not exist in your system. Check the list of available device names in the Dirs/Drives list box in the Open dialog box and use one of those.

Communication-buffer overflow

During remote communications, the receive buffer overflowed.

The size of the receive buffer is set by the RB option in the OPEN COM statement. To avoid this error, you can:

- Check the buffer more frequently using the LOC function.
- Empty the buffer more often using the INPUT$ function.

ERR code: 69

Permission denied

An attempt was made to write to a write-protected disk, or to access a locked file.

ERR code: 70

QBasic permission denied

You are attempting to overwrite a read-only file or there is a write-protect tab on the disk you are trying to write to.

To avoid the read-only conflict, use a different filename. Remove the tab or use a different disk if the write-protect tab is present.

Disk not ready

The disk-drive door is open, or no disk is in the specified drive. Insert a disk in the drive, close the drive door, and retry the operation.

ERR code: 71

Disk-media error

Disk-drive hardware has detected a physical flaw on the disk.

ERR code: 72

QBasic Disk-media error

QBasic does not recognize the format of the disk it is attempting to use. Use a different disk or exit QBasic and reformat the current disk.

Feature unavailable

You may be attempting to use a feature of another version of Basic that is not available with Microsoft QBasic.

ERR code: 73

Rename across disks

You attempted to rename a file with a new drive designation. You cannot rename a file with a new drive designation in QBasic.

ERR code: 74

Path/file access error

During an OPEN, MKDIR, CHDIR, or RMDIR operation, the operating system was unable to make a correct connection between the path and filename.

In the QBasic environment, make sure the file specification you entered in the text box is formatted correctly. Any filename can contain a drive name and a full or partial path.

This error can also occur when you try to save a file which would replace an existing read-only file.

ERR code: 75

Path not found

During an OPEN, MKDIR, CHDIR, or RMDIR operation, the operating system was unable to find the path specified.

ERR code: 76

QBasic Path not found condition

DOS was unable to find the path that you specified in the dialog box for the Open command, the Save command, or the Save As command. Check that you typed the path correctly.

Out of stack space

This error can occur when there are too many active

subroutine, FUNCTION, and SUB calls, or when a recursive FUNCTION procedure nests too deeply.

Change the structure of your program, or use the CLEAR statement to increase the program's allotted stack space.

Cannot continue

While paused during debugging (at a breakpoint, etc.), you made a change that prevents execution from continuing. You may have redimensioned an array, changed procedure arguments, or edited some other declarative statement. Use the dialog box and either:

- Choose <OK> to keep the change and restart the program.
- Choose <Cancel> to undo the change in your program text. (You can do this if restarting interferes with your debugging strategy.)

Duplicate procedure names in file

The file you attempted to open contains two SUB or FUNCTION procedures with the same name. Use an editor to rename one of them, then open your file in QBasic.

Overwrite existing file

Choose <Yes> only if you are sure that by overwriting the contents of the file on disk with the file contents in memory you won't lose valuable information. Otherwise:

- Choose <No> to redisplay the dialog box and enter a new, unique filename for saving the file in memory.
- Choose <Cancel> to stop the command, leaving everything as it was before you chose the command.

Delete procedure confirmation

Before deleting a procedure, the SUBs command verifies that you want to delete the procedure from the module. Either:

- Choose <OK> to delete the procedure.
- Choose <Cancel> to stop the deletion, leaving everything as it was before you chose the command.

Blank lines not allowed

Ordinarily the SUB or FUNCTION statement that begins a procedure is the first line in the View window. When you try to put a blank line above the SUB or FUNCTION statement, QBasic transforms the blank line into a comment line.

File not saved

Your file has either never been saved or has not been saved since it last changed. Either:

- Choose <Yes> to save the file, preserving your changes.
- Choose <No> to discard your changes.
- Choose <Cancel> to cancel the command, leaving everything as it was before you chose this command.

Cannot cross procedure boundary

During debugging, you used the Set Next Statement command to specify a statement that is in a procedure other than the one where program execution is suspended. In this case, QBasic ignores the Set Next Statement command.

To continue debugging, move the suspended procedure into the View window. You can then use the Set Next Statement command to start running from any statement in that procedure.

Specify name

You chose <OK> in a dialog box when the contents of the text box were blank. Type a module or procedure name in the text box and chose <OK> again.

DATA statements were moved

QBasic requires that all DATA statements in a program be in the module-level code. QBasic automatically moves DATA statements to the module level.

No search string

The Find What text box was empty when you executed a Find or Change command. Choose the command again and enter a text string before choosing <OK>.

Match not found

QBasic could not find the string specified.

The cursor remains at the location where you started the search.

Change complete

The search and replace task is complete. The cursor is back at the point where the search started.

SUB or FUNCTION required

You selected the module name from the SUBs command dialog box. Select a SUB or FUNCTION procedure instead.

Insert diskette

You have referred to your floppy disk drive as A: when it is named B:, or B: when it is named A:.

Make sure the file you want this command to operate on is on a disk in the floppy disk drive, then choose <OK>.

Cannot set Tabs

When you have a file open that contains a previously set tab stop, it cannot be changed with the Tab Stops option from the Display command. Save your file, then use the New command to clear the QBasic View window. Use the Display command to set the tab stop, then reopen your file.

Out of data space

QBasic ran out of room in memory for data, variables, and some other information. Try modifying your data space requirements as follows:

- Use a smaller file buffer in the OPEN statement's LEN clause.
- Use the $DYNAMIC metacommand to create dynamic arrays. Dynamic array data can usually be much larger than static array data.
- Use fixed-length string arrays instead of variable-length string arrays.
- Use the smallest data type that accomplishes your task. Use integers whenever possible.
- Use CLEAR to modify the size of the stack.
- Do not use source lines longer than 256 characters. Such lines require allocation of additional text buffer space.

Module-level code too large

The size of your module-level code exceeds QBasic's internal limit. Try moving some of the code into SUB or FUNCTION procedures.

Procedure too large

The size of the current procedure exceeds QBasic's internal limits. Make the procedure smaller by dividing it into several procedures.

Identifier too long

The indicated identifier is longer than QBasic allows. Identifiers, such as variable names, must not be longer than 40 characters.

Invalid identifier

You have used a character that is not valid in an identifier. Make sure that:

- DEF FN procedure names contain only letters and numbers.
- FUNCTION and SUB procedure names start with a letter and do not contain these special characters: ., %, &, !, #, or $.

Identifier cannot end with %, &, !, #, $

Identifiers cannot end with %, &, !, #, or $.

The above suffixes are not allowed in type identifiers or in SUB procedure names.

Identifier cannot include period

A period cannot be used in:

- The name of a user-defined data type.
- The name of an element in a user-defined type.
- The name of a variable dimensioned as a user-defined data type.

Although variable names can contain periods, it is recommended that a period be used only as a record variable separator. If you have programs that use a period in variable names, you could change them to use mixed-case instead. For example, variable ALPHA.BETA would become AlphaBeta.

Expression too complex

Certain internal limitations of QBasic have been exceeded.

For example, during expression evaluation, strings that are not associated with variables are assigned temporary locations. A large number of such strings can cause this error to occur. Likewise, a numeric expression made up of many complicated subexpressions can cause this error.

Try simplifying expressions or assigning strings to variables.

Illegal number

You have used a number that is inappropriate for the context in which it is used. For example, QBasic does not allow you to declare a fixed-length string of zero length, so DIM X AS STRING * 0 is illegal. An illegal number also occurs if you declare contradictory values. For example, using an integer data type declaration suffix with a

floating-point number (for example, 10.20%) causes this error.

Invalid constant

An invalid expression has been used to assign a value to a symbolic constant.

Numeric expressions assigned to symbolic constants can contain:

- Numeric literals.
- Previously defined symbolic constants.
- Any of the arithmetic or logical operators except exponentiation.

String expressions assigned to a symbolic constant may consist only of a single literal string, enclosed in double quotation marks.

Illegal outside of SUB, FUNCTION or DEF FN

QBasic has encountered a statement that is not allowed in module-level code. For example, EXIT SUB, EXIT FUNCTION, or EXIT DEF statements are not allowed in module-level code.

Illegal outside of SUB/FUNCTION

QBasic has encountered a statement that is not allowed in module-level code or DEF FN functions.

Illegal in SUB, FUNCTION, or DEF FN

The statement is not allowed inside a SUB or FUNCTION procedure or DEF FN function.

Statement cannot precede SUB/FUNCTION definition

Only comments and DEFtype statements are allowed before a procedure definition.

SUB/FUNCTION without END SUB/FUNCTION

The terminating statement is missing from a procedure.

END DEF without DEF

An END DEF statement has no corresponding DEF statement.

Element not defined

You have attempted to use an element that you have not defined previously as part of a user-defined data type.

For example, if you defined MYTYPE as containing elements A, B, and C, and you attempted to use the variable D as an element of MYTYPE, this message would appear.

Type not defined

A variable or parameter was declared with a undefined data type.

Use the TYPE statement to define a data type.

Must be first statement on the line

In block IF...THEN...ELSE constructs, IF, ELSE, ELSEIF, and END IF can only be preceded by a line number or label.

In SELECT...END SELECT constructs, CASE and END SELECT can be preceded only by a line number or label.

END SUB or END FUNCTION must be last line

You are attempting to add code after the last line in a procedure. You must either return to the main module or open another SUB or FUNCTION to add lines of code.

TYPE without END TYPE

There is no END TYPE statement associated with a TYPE statement.

END TYPE without TYPE

There is no TYPE statement associated with an END TYPE statement.

Statement illegal in TYPE block

Only comments and AS data type clauses are allowed between the TYPE and END TYPE statements.

Cannot start with 'FN'

You used "FN" as the first two letters of a SUB or a FUNCTION procedure or variable name.

"FN" can only be used as the first two letters when calling a DEF FN function.

Operation requires disk

You are attempting to load from, or save to, a nondisk device such as the printer or keyboard.

$Metacommand error

The syntax of a metacommand is incorrect.
Metacommands must be preceded by a comment. For example, both of the following $DYNAMIC metacommands are correct:

```
REM  $DYNAMIC
'  $DYNAMIC
```

Breakpoint not allowed on CASE clause

There are some restrictions on using breakpoints to debug SELECT...END SELECT statements. You cannot place a breakpoint on the first statement in the CASE block or after the END SELECT statement.

To debug a SELECT...END SELECT statement with the aid of a breakpoint, set the breakpoint on the SELECT CASE line in your program. Then single-step through the rest of the statement structure.

Array already dimensioned

This error can be caused by any of the following:

- More than one DIM statement for the same static array.
- An attempt to change the dimensions of a dynamic array with a DIM statement without using the ERASE statement to deallocate the array first.
- An OPTION BASE statement that occurs after an array is dimensioned.

Must precede executables

A COMMON statement or a DECLARE statement occurs after an executable statement. COMMON and DECLARE statements must appear before any executable statements. All Basic statements are executable except the following:

COMMON	OPTION BASE
CONST	REM
DATA	SHARED
DECLARE	STATIC
DEFtype	TYPE
DIM (for static arrays)	$STATIC and $DYNAMIC

END IF without Block IF

The IF keyword that starts an IF...ENDIF block is missing.

Also check other control structures within the IF...ENDIF block and verify that they are correctly matched. For example, a nested IF block without a matching ENDIF inside the outer IF...ENDIF block generates this error.

Block IF without END IF

There is no corresponding END IF in a block IF...THEN statement.

ELSE without IF

An ELSE clause appears without a corresponding IF. Sometimes this error is caused by incorrectly nested IF statements.

Also check other control structures within the IF...ENDIF block and verify that they are correctly matched. For example, a nested IF block without a matching ENDIF inside the outer IF...ENDIF block generates this error.

EXIT FOR not within FOR...NEXT

An EXIT FOR statement is used outside of a FOR...NEXT statement.

EXIT DO not within DO...LOOP

An EXIT DO statement is used outside of a DO...LOOP statement.

DO without LOOP

The terminating LOOP keyword is missing from a DO...LOOP statement.

LOOP without DO

The DO starting a DO...LOOP statement is missing or or misspelled.

Also check other control structures within the DO...LOOP structure and verify that they are correctly matched. For example, an IF block without a matching ENDIF inside the DO...LOOP structure generates this error.

SELECT without END SELECT

The terminating END SELECT keywords are missing from a SELECT CASE statement.

CASE without SELECT

The SELECT CASE keywords starting a SELECT CASE statement are missing or misspelled.

Also check other structures within the SELECT...END SELECT structure and verify that they are correctly matched. For example, an IF block without a matching END IF inside the SELECT...END SELECT structure generates this error.

END SELECT without SELECT

The SELECT CASE keywords beginning a SELECT CASE statement are missing or misspelled.

Also check other control structures within the SELECT...END SELECT structure and verify that they are correctly matched. For example, an IF block without a matching END IF inside the SELECT...END SELECT structure generates this error.

Fixed-length string illegal

You cannot use a fixed-length string as a formal parameter to a SUB or FUNCTION procedure.

Typed variable not allowed in expression

Variables of a user-defined data type cannot be passed as arguments. For example, the following expression, where X is a user-defined data type, is not permitted:

```
CALL ALPHA(X)
```

You can, however, pass elements of a user-defined data type as arguments, as follows:

```
CALL Alpha(X.FirstEl)
```

DEF without END DEF

There is no corresponding END DEF in a multi-line function definition.

Parameter type mismatch

A SUB or FUNCTION procedure parameter data type does not match the data type of the DECLARE statement's argument or the calling argument.

AS clause required on first declaration

A variable that has not been declared using an AS clause is referred to with an AS clause.

AS clause required

A variable declared with an AS clause is referred to without that clause.

If the first declaration of a variable has an AS clause, every subsequent DIM, REDIM, SHARED, or COMMON statement that refers to that variable must have an AS clause.

Illegal placement of statements

Statements and line labels are not permitted between SELECT CASE and the first CASE statement.

Comments and the statement separator (:) are permitted.

Numeric array illegal

Elements of numeric arrays are not allowed as arguments to VARPTR$.

Only simple variables and variable-length string array elements are permitted.

Illegal outside of TYPE block

The "element AS type" clause is permitted only within a TYPE...END TYPE block.

Wrong number of dimensions

An array reference contains the wrong number of dimensions.

DEF FN not allowed in control statements

DEF FN function definitions are not permitted inside control statements such as IF...THEN...ELSE and SELECT CASE.

FOR index variable already in use

This error occurs when the same variable is used as the counter argument in one FOR...NEXT statement and in a nested FOR...NEXT statement inside that loop. Nested FOR...NEXT statements must use diffrent variables for their counter arguments.

Type more than 65535 bytes

A user-defined data type cannot exceed 64K.